Store Less, Search Better: Access-Controlled Vector Search over Resolved Visibility

Chenyang Wu[1] Renxuan Wang[2] Zhongle Xie[1] Meihui Zhang[3] Gang Chen[1]
[1]Zhejiang University [2]Northwestern Polytechnical University [3]Beijing Institute of Technology
{cy.wu, xiezl, cg}@zju.edu.cn wangrenxuan@mail.nwpu.edu.cn meihui_zhang@bit.edu.cn

ABSTRACT

Vector databases increasingly underpin retrieval-augmented generation, recommendation, and multi-agent applications. In these systems, queries from different users may operate over different visible subsets of a shared collection. A shared index limits replication but may waste substantial search effort on invisible data, whereas isolated collection partitions provide cleaner search spaces at the cost of replicated storage. Yet existing partitioning methods are often designed around a particular authorization policy, which is expensive to rebuild as visibility evolves. We present SQUID, a policy-independent framework that performs physical design over resolved user-data visibility. SQUID groups vectors with identical visible-user sets into Access Atoms and places them into memory-bounded partitions. A Shared-Atom Graph and a recall-aware cost model balance storage savings against query latency. SQUID further allocates search effort according to route difficulty and enforces correct visibility before returning results. When data or visibility changes, it updates visibility immediately and repairs the affected search region. Experiments across multiple datasets and authorization policies show that SQUID achieves lower latency while using less memory than state-of-the-art methods.

PVLDB Reference Format:
Chenyang Wu, Renxuan Wang, Zhongle Xie, Meihui Zhang, Gang Chen. Store Less, Search Better: Access-Controlled Vector Search over Resolved Visibility. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/zjuDBxAI/SQUID.

1 INTRODUCTION

Vector databases increasingly support retrieval-augmented generation, recommendation, and multi-agent applications [18, 20, 37]. These deployments rely on a broad range of approximate nearest-neighbor (ANN) algorithms and systems, whose performance varies across datasets, workloads, and constraints [3, 7, 31]. In these applications, application-level access constraints give users different visibility over vectors embedded in a common semantic space. We call the resulting task *access-controlled vector search*: a query must retrieve approximate nearest neighbors only from the vectors visible to its user. A vector outside this valid search domain is unauthorized, or *invisible*, rather than an approximate answer, even if it is close to the query vector. The same vector may be visible to several users, and each user's visible vectors change as vectors and visibility are updated over time. The vector database must therefore account for different visible subsets in both storage organization and search execution to ensure efficiency.

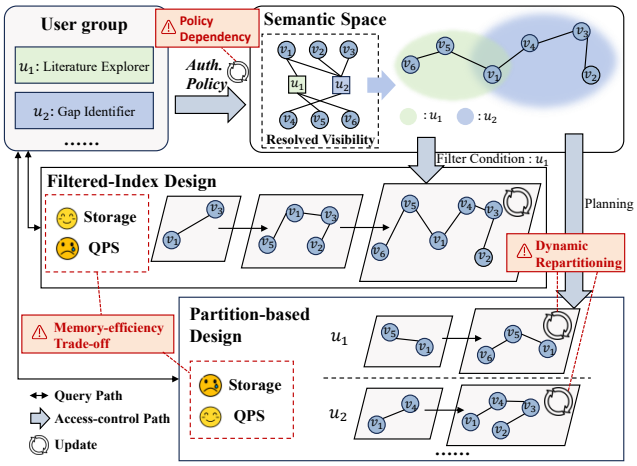


Figure 1: Access-controlled vector search in an AutoResearch agent system.

Figure 1 illustrates this problem in an AutoResearch agent system. Two users with roles, i.e., Literature Explorer and Gap Identifier, access overlapping but nonidentical sets of vectors embedded in the same semantic space. A neighborhood that is valid for one user to search may include vectors invisible to the other. Although the queries for different users use the same similarity measure, they induce different search spaces. External authorization policies, including role-based access control (RBAC) [8, 19, 30], attribute-based access control (ABAC) [5, 14], and discretionary access control (DAC) [1], determine which vectors each user may access. We call the resulting user–vector mapping *resolved visibility*.

This setting raises a database design question: *How should a vector database organize and search its stored vectors when each user has different visibility over them?* The central difficulty is that keeping storage compact and giving each user a focused search space favor different organizations. Keeping shared vectors together avoids storing them repeatedly, but a query may encounter many vectors that its user cannot access. Separating vectors by user visibility narrows each user's search space, but vectors may have replicas. Because visibility differs across users and changes over time, improving storage or search in isolation can make the other less efficient.

Existing systems have consequently moved visibility enforcement from query-time filtering toward the physical organization

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

of searchable data, as illustrated in Figure 1. Filtered-Index design maintains a shared index over the stored vectors and applies visibility constraints during search [12, 22, 27, 45]. Albeit compact in storage, filtered ANNS can incur substantial query computation on invisible vectors. The resulting computation on invisible vectors can increase end-to-end query latency and reduce throughput, particularly when a user's valid search domain is sparse.

To reduce this query-time filtering, partition-based designs further materialize visibility-aligned search spaces. Partition-based designs materialize smaller searchable partitions and route each query to partitions that may contain visible vectors [13, 22, 33, 43, 44]. By directing each user toward cleaner permitted search spaces, partitioning can avoid much of the invisible-vector work incurred by filtered ANNS. However, vectors visible to multiple users may be stored and indexed repeatedly. Partition planning therefore couples two decisions: which vector copies to retain in storage and which partitions each user must search. Moreover, when a user searches multiple partitions, the database must coordinate those searches and combine their candidates into one result. This evolution makes *partition planning*, which is the process of constructing partitions, vector copies, and user routes, i.e., the partitions searched for each user, a core design task for partition-based methods. Yet, existing state-of-the-art methods [13, 26, 44] face three challenges.

Challenge 1: policy-independent partition planning. Existing methods commonly perform partition planning over roles, attributes, or other authorization-policy objects, making the physical organization depend on the representation that produced resolved visibility. When these policy objects change, the derived partitions and indexes may need to be rebuilt. Prior work reports that such rebuilding can take hours on 1M vectors [44], making repeated construction an adaptation bottleneck. Planning separately for every vector removes this dependence but makes the decision space grow with the collection. Partition planning must therefore derive tractable decision units from resolved visibility without relying on either policy objects or individual vectors.

Challenge 2: balancing memory consumption and query computation. Partition-based methods face an inherent trade-off in how much user-specific isolation to materialize: greater isolation reduces computation on invisible vectors but replicates shared vectors, whereas sharing partitions among users saves memory but increases query computation. A vector copy or user route can change the search spaces of several users, so these decisions cannot be optimized per user or per partition. Moreover, planned partitions differ in size and in the fraction of vectors visible to each routed user, so the same query computation can yield different recall across partitions. Under a memory budget, partition planning must therefore compare each copy's memory consumption with the query computation it saves across the affected user–partition pairs at a common recall target.

Challenge 3: maintaining evolving partition organizations. Materializing the selected partitions, copies, and routes produces a *partition organization* whose indexes and metadata can become misaligned with users' current valid search domains as vectors or resolved visibility change. This misalignment can lower recall or increase query computation until the affected partitions and indexes are repaired. During repair, query execution must respect current resolved visibility and maintain the target recall, while copies, routes, partition contents, indexes, and metadata are updated consistently. Maintenance must therefore coordinate these repairs and restore query efficiency without making every change trigger database-wide repair.

We present SQUID, a framework that performs database-internal physical design over resolved visibility for access-controlled vector search. SQUID groups vectors with the same visible-user set into policy-independent *Access Atoms*. Starting from a selective per-user seed, its sparse Shared-Atom Graph exposes redundant Atom copies, and its recall-aware cost model ranks their consolidation under a memory budget. At query time, SQUID allocates partition-specific search effort, filters candidates under current resolved visibility, and merges authorized candidates across routed partitions. When vectors or visibility change, SQUID preserves query correctness while repairing the affected partitions and indexes.

Contributions. Our contributions are as follows:

- We formulate access-controlled vector search as a design problem over resolved visibility, and devise Access Atoms as policy-independent units connecting authorization semantics to partition planning, search and maintenance. (§2 & §3)
- We design a memory-bounded Partition Planner that exposes redundant copies through a Shared-Atom Graph and ranks their consolidation at a certain recall target. (§4)
- We develop a partition-aware Search Engine and a maintenance protocol that serves current visibility while repairing affected storage, indexes, and routing metadata. (§5)
- We evaluate SQUID's advance across multiple datasets, authorization policies, memory budgets, and update workloads against SOTA filtered-index and partition-based alternatives.(§6)

2 SYSTEM MODEL AND RETHINKING

In access-controlled vector search, each query is evaluated over the set of vectors visible to its user. Resolved visibility determines this valid search domain, while query efficiency depends on how the vector database organizes and searches the underlying vectors. This section first formalizes the system model, then revisits how resolved visibility shapes storage organization, index and search, and maintenance, and finally derives the design principles.

2.1 System Model

Let $U = \{u_1, \ldots, u_{|U|}\}$ and $D = \{d_1, \ldots, d_{|D|}\} \subseteq \mathbb{R}^h$ denote the user set and vector collection, respectively, where h is the vector dimensionality. The visibility of vectors to users is resolved by one or more authorization policies [5, 8, 14, 30]. We model the resolved visibility by the function $\mathcal{V} : D \to 2^U$, where $u \in \mathcal{V}(d)$ if and only if vector d is visible to user u. The visibility function $\mathcal{V}$ is a logical model; implementations that produce the same mapping from vectors to visible-user sets are equivalent under this model.

An access-controlled vector-search query is a tuple $Q = (q, u, k)$, where $q \in \mathbb{R}^h$ is a query vector, $u \in U$ is the querying user, and $k \in \mathbb{N}^+$ is the requested result cardinality. The valid search domain of Q is $D_u = \{d \in D \mid u \in \mathcal{V}(d)\}$. The search returns an approximate top-k result $R(Q) \subseteq D_u$ under the chosen distance metric. Approximation may affect which near neighbors in D_u are retrieved, but every returned vector must remain in D_u; a vector outside D_u is unauthorized rather than approximate.

The definitions of $\mathcal{V}$ and D_u specify query-visible vectors but leave the database realization open. A database must decide how vectors are organized into storage partitions, how ANN indexes are built over the partitions, how a query searches the resulting indexes, and how partitions, indexes, and auxiliary metadata are maintained against change [9, 10, 15, 17, 25, 32, 33]. Different realizations of the same $\mathcal{V}$ redistribute storage overhead, search computation, and maintenance work without changing the valid result domain D_u. The next subsection examines this design space in depth across storage organization, index and search, and maintenance.

2.2 Rethinking Design Space

To rethink this design space, we distinguish the planning process, its materialized organization, the indexes and search spaces that organization creates, and the maintenance required as visibility evolves. Note that *Partition planning* generates partitions, vector copies, and user routes, producing a *partition plan*. Materializing this plan produces a *partition organization* containing the stored copies, ANN indexes, routes, and associated metadata.

Storage organization. For partition planning, the core point is not which authorization-policy object grants access, but whether vectors have the same visible–user set. Vectors that are indistinguishable under resolved visibility can be grouped together without changing any user's valid search domain, reducing the complexity of the planning space. Physical decisions over these groups nevertheless remain coupled: removing a copy saves its storage once, but changes every user route that relied on it, and the resulting query computation depends on the partition into which its vectors are consolidated. The benefit of a copy is therefore determined by the surrounding partition organization rather than by itself alone.

Index and search. Partition planning changes not only the number of indexes searched by a user, but also the size and visible–vector fraction of each searched partition. These properties affect the recall obtained from a given amount of query computation. Consequently, evaluating alternative organizations at a fixed search depth can favor a consolidation merely because it attains lower recall. Their query costs must instead be normalized by the computation required to reach a recall target. Given the selected organization, query execution then distributes this computation across the routed indexes and filters their candidates under current visibility.

Maintenance. Visibility changes affect serving correctness and physical quality differently. A revoke can be enforced by filtering candidates under current visibility even while an existing index remains searchable, whereas a grant may require adding a reachable copy and route before the newly visible vectors can be found. The database must therefore update the affected copies, routes, and indexes consistently enough to serve the new valid search domains before optimizing their physical organization. Subsequent local reorganization can then remove redundant copies and recover recall and query efficiency within the affected region, avoiding reconstruction work proportional to the full database.

2.3 Design Principles

The three layers are coupled through visibility materialization. Storage decisions determine the indexes and search spaces exposed to each user; their query cost determines whether the memory consumed by a copy is justified; and visibility changes determine how long that organization remains useful and how much work is required to adapt it. Optimizing any layer in isolation can therefore shift cost to the other two. This fact yields three design principles for access-controlled vector search:

- **Policy-agnostic organization.** The database should derive tractable decision units from resolved visibility, avoiding dependence on both authorization-policy objects and vector-level decisions.
- **Benefit-driven materialization.** The planner should retain a materialized copy only when the query computation it saves across affected valid search domains at a common recall target justifies its memory and index overhead.
- **Resilient online maintenance.** Subsequent queries should observe current visibility while local repair restores recall and query efficiency in the affected partitions and indexes.

Table 1: Technical-route comparison.

Method	Storage organization		Index and search		Maintenance		Performance	
	Policy agnostic	Memory bounded	Recall-aware	Partition-aware	Delta update	Perf. repair	Mem. eff.	Query eff.
HONEYBEE [44]	×	✓	✓	✓	✓	×	Low	Low
VEDA/EFFVEDA [13]	×	✓	×	✓	✓	✓	Low	High
SQUID (Ours)	✓	✓	✓	✓	✓	✓	**High**	**High**

Discussion. As summarized in Table 1, representative state-of-the-art systems address complementary subsets of these three principles. They primarily follow a *policy-derived planning* route, using roles or role combinations to determine the partitions and copies considered during planning. HONEYBEE [44] exemplifies this route by optimizing role-derived partitions under memory and recall constraints. VEDA and EFFVEDA [13] couple memory-bounded organization with coordinated partition search and delta maintenance over role combinations. The resulting partitions and copies nevertheless remain tied to the policy representation, and maintaining a correct organization incrementally may not necessarily restore query performance after substantial drift. We instead follow a *visibility-driven planning* route: resolved visibility determines which vectors are equivalent for planning, copy benefits are measured at the same recall target, and the same equivalence localizes subsequent repair. The following sections define our abstraction and detail the solution.

3 SQUID

SQUID turns resolved visibility into a physical organization for access-controlled vector search. Its design centers on Access Atoms—groups of vectors with the same visible-user set. These Atoms provide a common unit across three stages: the Planner selectively replicates and consolidates them under a memory budget, the Search Engine searches the resulting partitions for each user, and the Maintainer repairs the affected state as vectors or visibility change. We first introduce this visibility-preserving unit and then follow it through SQUID's offline, query, and update paths.

3.1 Key Idea

Physical organization needs a unit that preserves visibility semantics without depending on authorization syntax: individual vectors

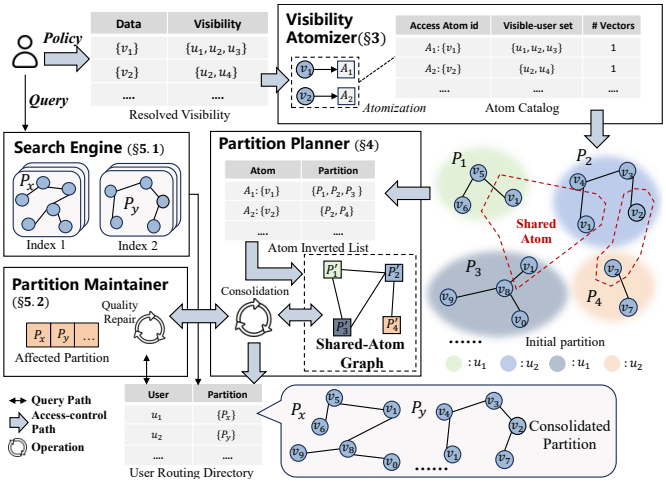


Figure 2: SQUID Overview

are too fine-grained for sharing detection, memory accounting, and maintenance, whereas roles or attributes bind the organization to the policy representation that produced visibility. SQUID therefore groups vectors by *shared visibility*. Under $\mathcal{V}$, vectors with the same visible-user set are indistinguishable for access-control correctness and form an Access Atom:

Definition 3.1. (Access Atom) An *Access Atom* for visible-user set $A_i \subseteq U$ is $S_{A_i} = \{d \in D \mid \mathcal{V}(d) = A_i\}$. All vectors in S_{A_i} are visible to exactly the same users. In Figure 2, v_1 belongs to the shared Atom of the Literature Explorer and Gap Identifier, while their private notes belong to different Atoms.

Discussion. Access Atoms are logical planning units rather than physical indexes. SQUID stores their copies in searchable *partitions*, where a partition may contain multiple Atoms and an Atom may appear in multiple partitions. Retaining copies yields smaller, cleaner user-visible search spaces; consolidating them saves memory but can create mixed partitions that require more filtering during search. Thus, unlike a global filtered index or policy-specific partitioning, Access Atoms expose the replication–search trade-off directly from resolved visibility and carry it consistently into planning, serving, and maintenance.

3.2 Architecture and Overview

Figure 2 organizes SQUID around the Access Atom Catalog produced from resolved visibility. The **Partition Planner** converts the catalog into memory-bounded Atom copies, searchable partitions, and per-user routes stored in the User Routing Directory. The Atom Inverted Table records the partitions containing each Atom, while the Shared-Atom Graph identifies partitions that share removable copies. The **Search Engine** consumes the routes, and the **Partition Maintainer** updates the same catalog, placement, routing, graph, and index state after vector or visibility changes.

Process and Data Flow. The three paths carry the same Atom-level state. Offline, the Atomizer converts resolved visibility into catalog entries, and the Planner produces memory-bounded partitions and records each user's route. In Figure 2, the shared Atom can initially be copied in both user-local partitions; the Planner subsequently decides whether to retain or consolidate these copies under the memory budget. At query time, the Search Engine follows the requesting user's route, searches its partitions, filters candidates against current visibility, and merges the authorized top-k. On an update, the Maintainer changes the affected visibility-equivalent group and repairs only the corresponding catalog, placement, route, graph, and index state rather than rebuilding the full organization.

Example. For the Literature Explorer u_1 and Gap Identifier u_2, the Atomizer places their shared vector v_1 in A_1 and keeps their remaining visibility groups as separate Atoms. The Planner consolidates the initial $P_1, \ldots, P_4$ into P_x and P_y; the User Routing Directory records the partitions to search for each agent. The Search Engine uses these entries to search authorized data, while the Maintainer updates the affected Atom, partition, and directory state when data or visibility changes.

4 PARTITION PLANNING

Before planning, the Visibility Atomizer groups vectors with identical resolved visible-user sets into Access Atoms and records them in the Access Atom Catalog. Given this catalog, the Partition Planner constructs a partition plan and user routes under a memory budget and target recall by exposing redundant copies, generating coverage-preserving consolidations, and ranking them with the recall-aware cost model in §4.4.

4.1 Partition-Planning Problem

The catalog specifies which vectors can move together, but not how their copies should be organized for search. The Partition Planner takes the catalog, a target Recall@k, and memory budget $\mathcal{B}$ as input. It must keep every visible vector reachable, keep the vectors stored across materialized partitions within $\mathcal{B}$, and minimize query computation at the target recall.

A *partition* P is a searchable unit containing Atoms $\mathcal{S}(P)$, with vector cardinality $|P|$; an Atom may be copied across partitions. A *partition plan* $\mathcal{P}$ comprises the partitions, vector-copy placements, and user routes generated by partition planning, and a user's *route* $\mathcal{P}(u) \subseteq \mathcal{P}$ is the set of partitions it searches. Every route must provide *route coverage*:

$$D_u \subseteq \bigcup_{P \in \mathcal{P}(u)} P, \qquad \forall u \in U. \tag{1}$$

Coverage ensures reachability, not route purity: a routed partition may contain Atoms invisible to its user, whose candidates are filtered during search. Planning therefore trades the memory saved by removing redundant copies against the extra query computation of searching mixed partitions.

Memory consumption. Partition planning is subject to an end-to-end memory limit covering vector storage, ANN indexes, and metadata. Because exact index and metadata overhead is unavailable before materialization, SQUID controls its dominant planning-dependent component—vector copies, i.e., physical occurrences of vectors in materialized partitions:

$$M(\mathcal{P}) = \sum_{P \in \mathcal{P}} \sum_{S_{A_i} \in \mathcal{S}(P)} |S_{A_i}|. \tag{2}$$

The Planner therefore enforces the vector-copy budget $\mathcal{B} = \beta|D|$, where $\beta \geq 1$, while §6.3 reports the realized end-to-end memory after materialization.

Query computation. Query computation is incurred whenever a user's route searches a partition. Let $C(u, P)$ denote the query computation required for u to search routed partition P at the target Recall@k. The plan-level query computation sums this cost over all user–partition pairs:

$$C(\mathcal{P}) = \sum_{u \in U} \sum_{P \in \mathcal{P}(u)} C(u, P). \tag{3}$$

§4.4 gives the concrete definition of $C(u, P)$; the abstract form above is sufficient for the planning problem.

*Definition 4.1 (Memory-Bounded Partition Planning (*MBP^2*)).* Given the Access Atom Catalog, memory budget $\mathcal{B}$, target Recall@k τ, and per-route query computation $C(u, P; \tau)$, MBP^2 is

$$\begin{aligned} \min_{\mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}} \quad & C(\mathcal{P}) \\ \text{s.t.} \quad & M(\mathcal{P}) \le \mathcal{B}, \\ & D_u \subseteq \bigcup_{P \in \mathcal{P}(u)} P, \quad \forall u \in U. \end{aligned} \tag{4}$$

Here, the target recall τ is an explicit input to MBP^2, and $C(u, P; \tau)$ denotes the per-route query computation at that target; we write $C(u, P)$ when τ is fixed.

The decision version of MBP^2 is NP-hard; Appendix C gives its formal statement and proof. Because globally coupled copy-removal choices do not scale to many Atoms and users, SQUID starts from a coverage-preserving seed and iteratively applies locally generated consolidations until the budget is met.

Running example. Figure 3 starts with four partitions containing two replicated Atoms. The Atom $A_1 = \{v_1\}$ is owned by P_1, P_2, P_3, while $A_2 = \{v_2\}$ is owned by P_2, P_4. These placements contribute three copies of A_1 and two copies of A_2 to $M(\mathcal{P})$. The following stages use their owner sets to expose candidate edges and select consolidations that preserve route coverage.

4.2 Shared-Atom-Based Candidate Generation

SQUID initializes planning with one partition $P_u^{(0)} = D_u$ and one route $\mathcal{P}^{(0)}(u) = \{P_u^{(0)}\}$ for each user u. This per-user seed provides maximally selective one-partition routes and exposes every shared Atom copy as an explicit consolidation choice. Because the seed may exceed the memory budget, the Planner subsequently removes copies, which is called consolidation, according to their query-computation penalty from a cost model. Appendix C.2 gives the corresponding copy accounting. The model, defined in §4.4, is recall-aware and estimates how a consolidation changes query computation at the target recall.

Only partitions sharing an Atom expose a removable copy, so SQUID avoids examining unrelated partition pairs. The Shared-Atom Graph $\mathcal{G} = (\mathcal{P}, E)$ represents these candidates, where an edge (P_x, P_y) stores shared-Atom set $\Gamma(P_x, P_y) = \mathcal{S}(P_x) \cap \mathcal{S}(P_y)$. Note that an edge identifies its shared copies but may leave the resulting partition plan to the consolidation process as described in §4.3. An edge identifies the shared Atoms and their duplicate placements at its two endpoints; the consolidation process in §4.3 determines the resulting partition plan.

For an Atom S_{A_i}, the Atom Inverted Table records its owner partitions as $\mathcal{I}(S_{A_i}) = \{P \in \mathcal{P} \mid S_{A_i} \in \mathcal{S}(P)\}$. A widely replicated

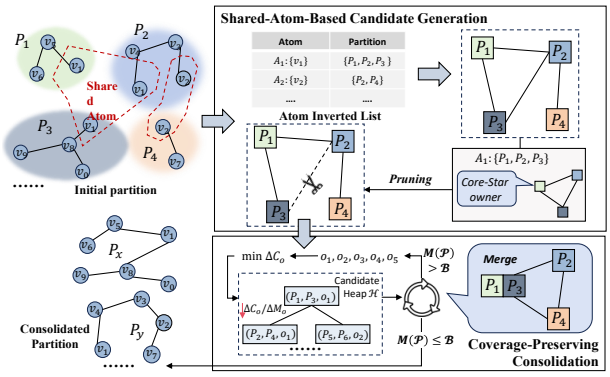


Figure 3: Partition Planning based on Shared-Atom Graph.

Algorithm 1 Shared-Atom-Based Candidate Generation

Require: Access Atom Catalog $\{S_{A_i} \mid A_i \in \mathcal{A}\}$, users U, pruning bound d
Ensure: Seed partition plan $\mathcal{P}$, routes $\{\mathcal{P}(u)\}_{u \in U}$, inverted table $\mathcal{I}$, candidate graph $\mathcal{G}$

$(\mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}, \mathcal{I}) \leftarrow \text{PerUserSeed}(catalog, U)$
$E_{\text{raw}} \leftarrow \emptyset$
for all $A_i \in \mathcal{A}$ with $|\mathcal{I}(S_{A_i})| > 1$ **do**
  $P_c \leftarrow \text{core}(S_{A_i})$ ▷ Eq. (5)
  for all $P \in \mathcal{I}(S_{A_i}) \setminus \{P_c\}$ **do**
    $\Gamma(P_c, P) \leftarrow \mathcal{S}(P_c) \cap \mathcal{S}(P)$
    $E_{\text{raw}} \leftarrow E_{\text{raw}} \cup \{(P_c, P)\}$
  end for
end for
for all $P \in \mathcal{P}$ **do**
  $E_P \leftarrow \text{TopD}(\{e \in E_{\text{raw}} \mid P \in e\}, \phi, d)$
end for
$\mathcal{G} \leftarrow (\mathcal{P}, \bigcup_{P \in \mathcal{P}} E_P)$
return $(\mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}, \mathcal{I}, \mathcal{G})$

Atom can make $\mathcal{G}$ dense. That is, connecting all $r = |\mathcal{I}(S_{A_i})|$ owners would create an $O(r^2)$ clique. To optimize, SQUID selects a *core-star owner* and connects it to every other owner, forming a core star. The core-star owner is defined as

$$\text{core}(S_{A_i}) = \arg \min_{P \in \mathcal{I}(S_{A_i})} \left(|P|, -|S_{A_i}|/|P|\right). \tag{5}$$

The core star reduces the Atom's candidate edges to $O(r)$, while omitting owner-pair candidates from the full clique. Choosing the smallest owner (i.e., the owner whose partition contains the fewest vectors) favors a compact anchor for the shared copy, while the tie-break favors the owner in which that Atom dominates the partition and is therefore less mixed with other Atoms.

When several Atoms induce the same edge, their witnesses accumulate in $\Gamma(P_x, P_y)$. SQUID ranks core-star edges with the lightweight overlap score

$$\phi(P_x, P_y) = \frac{|\Gamma(P_x, P_y)|}{\sqrt{|\mathcal{S}(P_x)|\,|\mathcal{S}(P_y)|}}, \tag{6}$$

The numerator counts shared Atoms, while the denominator discounts pairs whose overlap is small relative to their Atom sets. SQUID retains each partition's top-d raw incident edges, where $d \ge 1$, under this inexpensive pruning score and applies the full recall-aware cost model only to the retained candidates. Appendix C.3 details this sparsification rule, and Appendix E analyzes its time and space costs. Algorithm 1 summarizes this candidate-generation procedure.

Running example continued. As can be seen in Fig 3, for A_1, the core star selects P_1 and retains (P_1, P_2) and (P_1, P_3), while omitting (P_2, P_3). The two owners of A_2 contribute (P_2, P_4), so the graph exposes the illustrated consolidation opportunities without materializing the full owner clique.

4.3 Coverage-Preserving Consolidation

An edge locates a removable copy, but it does not determine where the surviving copy should reside. The memory saving is fixed by removing the duplicate, whereas keeping that copy with either residual side, isolating it, or reorganizing both residual sides creates different partition sizes and mixtures for the affected routes. The Planner therefore generates local alternatives and compares their query computation.

Given an edge $(P_x, P_y) \in E$, the Planner separates its endpoint partitions into the shared Atom set Ω and residual partition contents P'_x and P'_y: $\Omega = \mathcal{S}(P_x) \cap \mathcal{S}(P_y)$, $P'_x = \mathcal{S}(P_x) \setminus \Omega$, and $P'_y = \mathcal{S}(P_y) \setminus \Omega$. The shared Atom set is the source of released memory, while the residual partition contents determine the composition, and hence the selectivity, of the resulting route partitions.

Each of the following five alternatives replaces the two copies of Ω in the input partitions with one copy: absorption keeps it with one residual partition content, isolation places it in a separate partition, and reorganization either combines both residual partition contents with Ω or separates them from it:

$$\begin{array}{llllll} o_1 & : & \{P'_x \cup \Omega \cup P'_y\}, & o_2 & : & \{P'_x \cup P'_y,\ \Omega\} \quad \text{(reorganize)} \\ o_3 & : & \{P'_x,\ \Omega,\ P'_y\}, & & & \quad \text{(isolate)} \\ o_4 & : & \{P'_x,\ P'_y \cup \Omega\}, & o_5 & : & \{P'_x \cup \Omega,\ P'_y\} \quad \text{(absorb)} \end{array} \tag{7}$$

These alternatives exhaust the distinct ways to retain one copy of Ω while joining or separating the two residual Atom sets; empty residuals are omitted. For each affected user, the Planner replaces the two endpoint routes with output partitions covering the same Atoms and leaves unrelated route entries unchanged. The alternatives therefore preserve route coverage and authorization safety, but induce different partition sizes, mixtures, and query computation at the target recall. Let $\mathcal{P}_o$ denote the partition plan obtained by applying operation o to (P_x, P_y). SQUID considers operations that release memory, $\Delta M_o = M(\mathcal{P}) - M(\mathcal{P}_o) > 0$ and measures their change in query computation as $\Delta C_o = C(\mathcal{P}_o) - C(\mathcal{P})$. The Planner first prioritizes operations with non-positive ΔC_o, breaking ties by larger ΔM_o and then a fixed operation order. If more memory must be released, it orders the remaining operations by $\Delta C_o / \Delta M_o$, the computation increase per unit of saved memory. For instance, absorption can place Ω in a larger mixed partition, whereas isolation can retain a cleaner shared partition while releasing the same copy; their different route selectivities yield different ΔC_o. Appendix C.5 gives the exact local memory and cost accounting.

Algorithm 2 summarizes the consolidation loop.The Planner repeatedly applies the best valid heap entry, updates only its affected partitions, routes, Atom Inverted Table entries, and graph edges, and rebuilds the heap when no valid entry remains. Appendix C.6 specifies candidate invalidation and local refresh. When $\mathcal{B} \geq |D|$ and $d \geq 1$, an over-budget plan contains a multiply owned Atom, so rebuilding exposes a consolidation that releases a copy and allows the loop to continue toward feasibility.

Algorithm 2 Coverage-Preserving Consolidation

Require: Planning $\mathcal{P}$, routes $\{\mathcal{P}(u)\}_{u \in U}$, graph $\mathcal{G}$, Atom Inverted Table $\mathcal{I}$, pruning bound d, memory budget $\mathcal{B}$, target recall τ
Ensure: Coverage-preserving partition plan and routes

$\mathcal{H} \leftarrow \text{BuildHeap}(\mathcal{G}, \mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}, \tau)$
while $M(\mathcal{P}) > \mathcal{B}$ **do**
  $o \leftarrow \text{PopBestValid}(\mathcal{H})$
  if $o = \perp$ **then**
    $(\mathcal{G}, \mathcal{H}) \leftarrow \text{RebuildCandidates}(\mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}, \mathcal{I}, d, \tau)$ ▷ Rebuild from the current Atom Inverted Table
  else
    Apply o and replace the affected routes using the coverage-preserving rule
    Update $\mathcal{P}$, $\{\mathcal{P}(u)\}_{u \in U}$, $\mathcal{I}$, and the incident edges of $\mathcal{G}$
    $\text{RefreshCandidates}(\mathcal{H}, \mathcal{G}, \mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}, \mathcal{I}, d, \tau)$
  end if
end while
return $(\mathcal{P}, \{\mathcal{P}(u)\}_{u \in U})$

Running example continued. For (P_1, P_3), the shared set is $\Omega = \{A_1\}$, with residual vectors $\{v_5, v_6\}$ and $\{v_9, v_8, v_0\}$. The illustrated o_1 operation combines them into a single output partition P_x. Specifically, $P_x = \{v_5, v_6, v_1, v_9, v_8, v_0\}$.This replaces two copies of A_1 with one and releases $\Delta M_{o_1} = |A_1|$.Likewise, applying o_1 to (P_2, P_4) yields $P_y = \{v_4, v_3, v_1, v_2, v_7\}$. Together, these transformations retain A_1 in P_x and P_y, consolidate A_2 into P_y, and update the affected routes while preserving coverage.

4.4 Recall-Aware Cost Model

The recall-aware cost model compares local alternatives at a common Recall@k target.For a routed user–partition pair (u, P), let $N = |P|$ and define its *route selectivity* $\rho(u, P) = \frac{|D_u \cap P|}{|P|}$, the fraction of vectors in P visible to u.Because N and $\rho(u, P)$ affect recall at a fixed HNSW [25] search depth ef_s, SQUID compares routes by the query computation required to attain the common target τ.

For route (u, P), $x = \frac{ef_s \cdot \rho(u,P)}{k}$ is the effective visible-candidate budget per requested result. A useful planning estimator must increase with search depth and saturate as recall approaches one; to retain the same recall, a larger partition or lower route selectivity must be offset by a larger ef_s. A Hill-type saturation function [39] provides these properties by mapping the effective visible-candidate budget to a bounded recall curve with diminishing returns:

$$\widehat{\text{Rec}}(N, k, ef_s, \rho) = \frac{x^\gamma}{\lambda(N)^\gamma + x^\gamma}, \quad \lambda(N) = \lambda_0 + \lambda_1 \log(1 + N). \tag{8}$$

Here, $\gamma > 0$ controls how quickly recall saturates, while $\lambda(N)$ captures the logarithmic size effect; a lower route selectivity reduces x at fixed search depth. The parameters $\gamma, \lambda_0, \lambda_1$ are fitted offline, with $\lambda_1 > 0$ and $\lambda(N) > 0$ over the planned partition sizes. For fixed N, k, $\rho > 0$, and $x > 0$, $\frac{d\widehat{\text{Rec}}}{dx} = \frac{\gamma \lambda(N)^\gamma x^{\gamma-1}}{(\lambda(N)^\gamma + x^\gamma)^2} > 0$, More precisely, $\widehat{\text{Rec}}(N, k, ef_s = 0, \rho) = 0$, and $\lim_{ef_s \to \infty} \widehat{\text{Rec}}(N, k, ef_s, \rho) = 1$. Together with monotonicity in ef_s, these limits make the model invertible at a common recall target.

Solving this monotone function gives the search depth required for a common target $\tau \in (0, 1)$:

$$\widehat{ef}_s(N, k, \tau, \rho) = \frac{k \lambda(N)}{\rho} \left(\frac{\tau}{1 - \tau} \right)^{1/\gamma}. \tag{9}$$

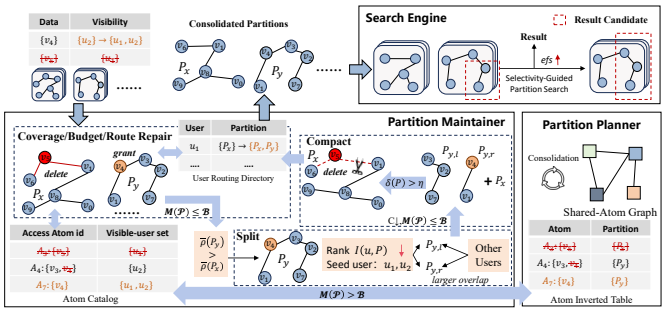


Figure 4: Query processing and maintenance.

Algorithm 3 Selectivity-Guided Partition Search

Require: $Q = (q, u, k)$ (query), D_u (visible set), $\mathcal{P}(u)$ (route), ef_0 (probe depth), dist (metric)
Ensure: R, authorized approximate global top-k over D_u; TopK_k retains the k closest candidates

$\rho(u,P) \leftarrow |D_u \cap P|/|P|$; order $\mathcal{P}(u)$ by decreasing $\rho(u,P)$
$R \leftarrow \emptyset$; $\delta_k(R) \leftarrow$ its k-th distance (or $+\infty$ if $|R| < k$)
for all $P \in \mathcal{P}(u)$ **do**
  $C_0(P) \leftarrow \textsc{Search}(P, q, ef_0)$; $A_0(P) \leftarrow C_0(P) \cap D_u$
  if $\exists x \in A_0(P) : \mathrm{dist}(q,x) < \delta_k(R)$ **then**
    $\rho_0(P) \leftarrow |A_0(P)|/|C_0(P)|$; $ef_P \leftarrow \lceil ef_0/\rho_0(P) \rceil$
  else if $A_0(P) = \emptyset$ **then**
    $ef_P \leftarrow \lceil ef_0/\rho(u,P) \rceil$
  else continue
  end if
  $R \leftarrow \mathrm{TopK}_k\big(R \cup (\textsc{Search}(P, q, ef_P) \cap D_u)\big)$
end for
return R

Following prior cost-modeling practice in vector search and database systems [22, 28, 44], SQUID uses an offline-fitted latency surrogate to estimate the per-route query computation $C(u,P)$ from this depth:

$$C(u,P) = a\log(1+N) + c_0 + b\log(1+N)\cdot\widehat{ef}_s(N,k,\tau,\rho(u,P)), \quad (10)$$

where a, b, and c_0 are offline-fitted latency-surrogate coefficients; SQUID does not introduce a new latency-training procedure, but supplies the recall-normalized search depth used by this surrogate. The three terms capture graph navigation, per-partition overhead, and target-recall search effort, respectively. This model is a planning estimator for comparing candidate partition plans at a common recall target, rather than a query-level recall guarantee. The evaluation therefore validates its plan-ranking calibration separately and reports query performance at measured recall. Summing the per-route estimates over routed user–partition pairs yields $C(\mathcal{P})$, which determines ΔC_o and the candidate-heap ordering. Appendix B details the recall and latency-surrogate calibration procedure.

5 QUERY PROCESSING AND MAINTENANCE

In this section, we describe how SQUID controls partition routing and search depth during search, and how it performs incremental updates to maintain partition quality under dynamic workloads.

5.1 Search over Partitions

Partition planning reduces replication by allowing a user route to include mixed partitions, but the resulting routes can differ substantially in the fraction of the vectors visible to the querying user. A uniform search depth may therefore waste query computation on high-selectivity routes or offer insufficient exploration for low-selectivity routes. For a query $Q = (q, u, k)$, the Search Engine obtains $\mathcal{P}(u)$ from the User Routing Directory, distributes query computation across the routed partitions, filters vectors invisible to u, and returns approximate top-k over the valid search domain D_u.

Index setting. Each materialized partition is served by an underlying ANN index. SQUID uses HNSW [25] by default, but its routing and query-computation allocation are independent of this choice; a partition may instead use any filtered ANN index such as ACORN [27]. The chosen index generates candidates within a partition, while the Search Engine selects routes, distributes query computation, filters unauthorized candidates, and merges authorized candidates across partitions.

Search Process. Algorithm 3 summarizes the search flow. It visits routed partitions in decreasing selectivity, uses an initial search to observe their authorized candidates, expands competitive or inconclusive partitions with a partition-specific depth, and merges the resulting authorized candidates into the final top-k result.

The algorithm adopts an established adaptive probing pattern, but operates at the visibility-route level rather than treating probing itself as a new ANNS primitive. First, graph-based filtered ANNS methods adapt candidate generation within one index [12, 27], whereas SQUID coordinates multiple partitions already selected by routing based on resolved visibility. Second, SQUID allocates partition-specific query computation using the authorized yield observed for the current query and the evolving global top-k result, rather than expanding solely according to geometric proximity or a static predicate selectivity. Third, an empty authorized probe is inconclusive, so SQUID falls back to cataloged route selectivity instead of discarding the routed partition.

It is worth noting that SQUID does not use the recall-aware cost model from §4.4 during query execution. That model compares candidate partition plans offline, whereas query execution can use current observations without evaluating and inverting the model for each partition. Appendix A.1 details the search-depth allocation, and Appendix A.2 formalizes how the two heuristics remain consistent in partition-planning-level expected query computation: the offline heuristic uses its estimate to compare candidate organizations, while the online heuristic adapts partition-specific search depths within the selected organization.

5.2 Update Maintenance

Vector and resolved-visibility updates can affect three properties of the current partition organization. We call a visibility addition a *grant* and a visibility removal a *revoke*. First, a revoke or deletion must immediately prevent an ineligible vector from being returned. Second, a grant, insertion, or change in Access Atom membership may require a new Atom copy, ANN index, and route before every newly visible vector becomes reachable. Third, these physical changes may exceed the memory budget, reduce route selectivity, or leave deleted vectors physically present. SQUID therefore makes the changed visibility effective for filtering first, restores route coverage and memory feasibility in a publishable serving state, and then repairs query efficiency only within the affected region.

The Partition Maintainer performs this recovery in the background while the Search Engine continues serving queries from the

latest published partitions, indexes, and routes. The Search Engine applies the latest visibility during candidate filtering, whereas a new Atom copy and route become visible to queries only after the corresponding partition and ANN index are ready. Appendix D formalizes the published-state invariants, and Appendix D.2 details this batch transition and publication order.

Coverage Repair. Coverage Repair updates the Access Atom Catalog and Atom Inverted Table, records deletions in their current partitions, reconstructs changed Atom copies, and marks the affected User Routing Directory entries for route repair. A revoke operation is immediately enforced by filtering, whereas a newly created or enlarged Atom requires a searchable copy and repaired route to become reachable. Coverage Repair constructs the copy state required to cover these newly visible Atoms; the resulting copies determine whether subsequent local repair must release copies or may spend residual memory to recover selectivity.

Budget Repair. Local repair first restores memory feasibility and uses only residual memory to recover query efficiency. Let $\mathcal{P}_\Delta$ denote the directly changed and newly assigned partitions, and define their one-hop repair region as

$$\mathcal{P}_\Delta^+ = \mathcal{P}_\Delta \cup \mathcal{N}_G(\mathcal{P}_\Delta), \tag{11}$$

where $\mathcal{N}_G(\mathcal{P}_\Delta)$ contains partitions connected to $\mathcal{P}_\Delta$ by retained Shared-Atom Graph edges. If $M(\mathcal{P}) > \mathcal{B}$, where $\mathcal{P}$ is the current partition plan after coverage repair, the Maintainer invokes the Partition Planner on the updated Atom and copy state within $\mathcal{P}_\Delta^+$ and processes consolidations that remove redundant copies until the memory bound is restored. Once the plan is feasible, any residual copy budget may be used by a *Split* operation, which divides a mixed partition into more selective outputs among partitions in $\mathcal{P}_\Delta^+$. Partitions and indexes outside this region remain unchanged, and routes that cover no changed Atom retain their entries. Appendix D gives the repair-region and route-coverage details.

Split. Split uses residual copy budget to reduce filtering in poorly selective mixed partitions. Let $U(P) = \{u \mid P \in \mathcal{P}(u)\}$ denote the users routed to P. To select a partition, SQUID considers mixed partitions $P \in \mathcal{P}_\Delta^+$ with $|U(P)| \geq 2$ in increasing average route selectivity $\bar{\rho}(P) = \frac{1}{|U(P)|}\sum_{u\in U(P)} \rho(u,P)$, breaking ties by lower $\min_{u\in U(P)} \rho(u,P)$ and then larger $|P|$. Within the selected partition, the filtering impact $I(u,P) = |D_u \cap P|\big(1-\rho(u,P)\big)$ prioritizes users whose queries discard the most candidates. Among these users, SQUID chooses as seeds the pair minimizing the vector-weighted overlap $\omega_P(u,v) = \frac{|P\cap D_u \cap D_v|}{|P\cap (D_u \cup D_v)|}$, thereby separating users with the least overlapping visible vectors. Each remaining user, processed in decreasing $I(u,P)$, joins the seed group with which it shares more vectors through common visible Atoms; ties favor the group with the smaller total filtering impact. An Atom visible to users in both groups is replicated into both output partitions, whereas every other Atom is placed only in the output for the group containing users who can see it. Replacing P with these outputs and repairing the affected routes yields a coverage-preserving candidate plan $\mathcal{P}'$. SQUID accepts the split only if $C(\mathcal{P}') < C(\mathcal{P})$ and $M(\mathcal{P}') \leq \mathcal{B}$.

Route Repair. The accepted Atom copies determine the final route repair. For each affected user, the Maintainer retains a complete route when possible; otherwise, it greedily adds the partition containing the most uncovered Atoms, preferring higher route selectivity and smaller size on ties. The repaired routes satisfy $D_u \subseteq \bigcup_{P\in\mathcal{P}(u)} P$ for every $u \in U$. Appendix D gives the corresponding route-repair and split semantics.

Compaction and Batch Completion. After the partition plan and repaired routes are fixed, the Maintainer reclaims physically retained deleted vectors through compaction without changing Atom copies or user routes. For partition P, let $d(P)$ count vectors recorded for deletion but still physically present and let $|P|$ count its valid vectors, giving $\delta(P) = \frac{d(P)}{|P|+d(P)}$. SQUID compacts P when $\delta(P) \geq \eta$, where η is a configurable compaction threshold [28]. The Maintainer then materializes every new, changed, or compacted partition from its valid vectors and builds its ANN index before publishing the corresponding routes and metadata. Consequently, revokes take effect immediately, while newly visible vectors become reachable after their background-built copies, indexes, and routes are published. The batch completes after the compacted state is saved, while unrelated partitions, indexes, and routes remain available without reconstruction. Appendix D further specifies compaction and publication semantics.

Running Example. Figure 4 applies a batch that deletes v_5 and grants u_1 access to v_4. The Search Engine immediately excludes v_5 under the changed visibility, while Coverage Repair records its deletion and places the new Atom containing v_4 in P_y. Because the resulting plan remains within the memory budget, SQUID skips consolidation and considers splitting P_y only if the split reduces $C(\mathcal{P})$ within the residual budget. Route repair then extends u_1's route to P_y, and v_4 becomes reachable after the corresponding index and route are published. Finally, the Maintainer removes v_5 from P_x through compaction when $\delta(P_x) \geq \eta$.

6 EVALUATION

Our evaluation answers four research questions:

RQ1. Authorization-policy independence. Can SQUID construct a partition plan from resolved visibility and query its materialized partition organization without relying on the policy representation that produced it? (§6.2.3)

RQ2. Memory–performance trade-off. Within a fixed memory budget, how does SQUID trade memory consumption for query performance relative to shared, fully isolated, and budget-constrained partition organizations? (§6.2.1, §6.2.2 & §6.3)

RQ3. Planning and maintenance. How much time does SQUID need to construct a partition plan and maintain its partition organization, and how does QPS change across update batches? (§6.4)

RQ4. Mechanisms and robustness. How much do the Search Engine and recall-aware cost model contribute, how sensitive are planning and query performance to the main parameters, and does SQUID remain compatible with a filtered ANN index? (§6.5)

6.1 Experimental Setup

We evaluate both partition planning and execution of the resulting partition organization in a real database system. Planning experiments measure the replication budget, materialized database footprint, planning time, and maintenance time, whereas query experiments measure Recall@k, end-to-end QPS, and single-query time after routing, ANN search, authorization filtering, and result merging.

Table 2: Datasets and visibility workloads. Avg. fan-out: authorized users per vector; ROLE/RLS: full-ROLE footprint relative to one-copy RLS.

Dataset	Dim.	Size
SIFT [23]	128	1M
Wiki [35]	768	485K
YFCC [34]	192	1M

Visibility Workload	Avg. Fan-out	Avg. Sel.	ROLE/ RLS
Tree-based [36, 44]	36.1	3.61%	~ 3.5×
ERBAC [21, 44]	69.4	6.94%	~ 6.2×
YFCC-ABAC [14, 30]	13.4	1.34%	~ 25.7×

Table 3 High-recall QPS and memory on SIFT+Tree-based; memory ratios are normalized to SQUID.

Method	#Partitions	Mem.(MB)	Mem.(×)	QPS
HONEYBEE	77	4236.000	2.002	2752.000
EFFVEDA	100	3968.555	1.875	3240.990
VEDA	97	3933.430	1.859	3355.691
SQUID	**58**	**2116.008**	**1.000**	**3545.451**

Table 4 Candidate-degree sensitivity (default in bold).

Top-d	Query Time (ms)	Planning Time (s)
4	3.663	2.020
8	4.945	2.188
16	3.149	2.064
32	**4.163**	**2.083**
64	3.468	2.157

Platform. All experiments run in a Docker container with 16 CPU cores and 256 GB of memory, PostgreSQL 17 [28], and pgvector 0.8.0. Unless stated otherwise, all methods use HNSW [25] and the same database configuration, query sequence, and warm-up procedure.

Datasets and visibility workloads. Table 2 summarizes the three datasets and their paired visibility workloads detailed below.

- ERBAC [21, 44] uses 40 functional and 100 business roles, with at most three functional roles per business role and three business roles per user.
- Tree-based [36, 44] uses 100 roles in a height-4 hierarchy with 3–4 children per internal role, producing nested visible-vector sets.
- YFCC-ABAC [14, 30] uses 50 two-label attribute rules derived from YFCC metadata, with two rules per user, overlap probability 0.5, minimum support 16, and 256 candidate clauses per rule.

Baselines. We compare SQUID with the following baselines:

- RLS uses one shared HNSW index and enforces resolved visibility through PostgreSQL Row-Level Security during query execution.
- ROLE Partition materializes a role-local search space and serves as an unconstrained isolation reference when its replication exceeds the budget.
- HQI [26], HONEYBEE [44], VEDA [13], and EFFVEDA [13] are policy-aware partitioning methods that trade replication for lower query computation.
- ACORN [27] is a filtered ANN index used alone and within SQUID's mixed partitions.

ERBAC and Tree-based provide native roles, whereas for YFCC-ABAC each user is represented as a virtual role with the same visible-vector set, giving role-oriented baselines 1,000 virtual roles.

Queries and metrics. All methods receive the same resolved visibility and query set, with exact visible top-k results within D_u as ground truth; $k = 10$ unless stated otherwise. Each QPS point reports the mean of five repetitions of 1,000 sampled queries after one warm-up round. QPS measures end-to-end throughput through PostgreSQL, whereas query time measures one SQL query to exclude client-side concurrency and isolate routing and search over the materialized partition organization. For each fixed partition organization, we sweep search depth and report the smallest depth reaching each Recall@k target.

Common protocol and parameter tuning. We express the planning budget as the ratio of materialized vector copies to input vectors; a 1.5× budget permits at most 1.5 copies per input vector on average. The realized database footprint, including vector relations, per-partition ANN indexes, and planning metadata, is measured separately in §6.3. We build SQUID, VEDA, and EFFVEDA on the released HONEYBEE codebase and reuse its implementations of RLS, ROLE Partition, HQI, and ACORN. HNSW-based methods use L_2 distance, $M = 16$, and $ef_{\text{construction}} = 64$. A method that exceeds the requested replication budget or the three-hour construction limit is omitted from query evaluation.

6.2 Memory–Efficiency Trade-offs

Figure 5 reports end-to-end QPS and single-query execution time for five dataset–workload pairs under the common 1.5× vector-copy budget. ROLE, as the upper bound, represents unconstrained full isolation, while RLS represents one shared index as a filtered-index design. The remaining methods are evaluated under the common budget. Each curve sweeps search depth for a fixed partition plan, and a missing curve indicates that the method did not produce a budget-feasible plan within three-hour partition construction.

6.2.1 Budget-Constrained Query Performance. Across the dataset–workload pairs in Figure 5, SQUID forms the strongest high-recall QPS and query-time curves among methods constrained to 1.5× vector copies. Near 0.95 recall on SIFT+ERBAC, SQUID provides 1.29× the QPS of EFFVEDA and 2.05× that of HONEYBEE. On Wiki+Tree-based, it improves QPS by 13% over HONEYBEE, 25% over EFFVEDA, and more than 8× over RLS. On SIFT+Tree-based, SQUID reduces matched-recall query time by 24%–37% relative to HONEYBEE, VEDA, and EFFVEDA and by approximately 7× relative to RLS. On YFCC+ABAC, it reaches 4.7× EFFVEDA's QPS and reduces its query time by 3.8×. These results show that SQUID's advantage persists across image and text vectors and across role- and attribute-derived visibility.

6.2.2 Cost of Full Isolation. ROLE bounds the performance gap to full isolation but exceeds the 1.5× budget. Under Tree-based visibility, SQUID uses 57% fewer vector copies and remains within 1.26× of ROLE's latency at 0.95 recall. Under ERBAC, SQUID uses 75% fewer copies and remains within 1.84× of ROLE's matched-recall latency. Closing the remaining latency gap therefore requires ROLE to store 2.3×–4.0× as many vector copies as SQUID.

6.2.3 Authorization-policy Independence. Tree-based and ERBAC derive visibility from role structures, whereas YFCC-ABAC derives it from attribute rules. SQUID consumes the same resolved-visibility representation and runs the same Planner without policy-specific configuration in all three cases. For fairness, policy-oriented baselines receive native roles for Tree-based and ERBAC and 1,000 visibility-equivalent virtual roles for YFCC-ABAC. On YFCC-ABAC, HONEYBEE, VEDA, and HQI exceed the construction limit, while

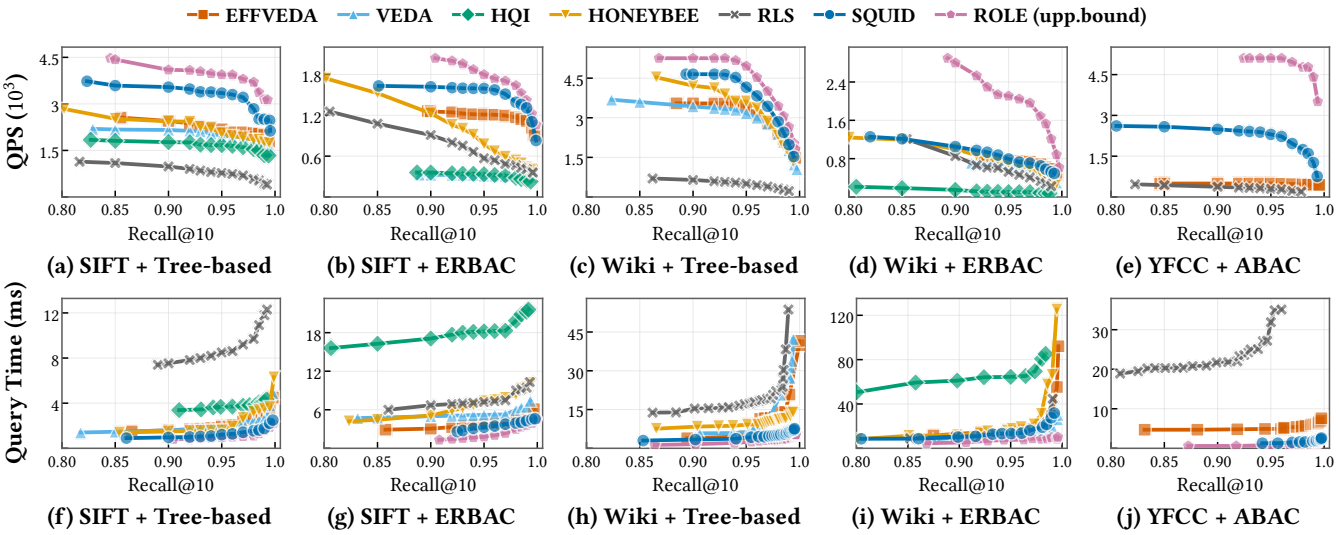


Figure 5: QPS–Recall@10 (top) and query-time–Recall@10 (bottom) across five dataset–workload pairs. Constrained methods use 1.5× copies; ROLE is unconstrained and fully materialized.

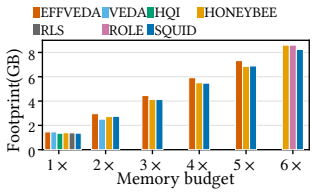


Figure 6: Footprint across copy budgets; missing bars denote unavailable settings.

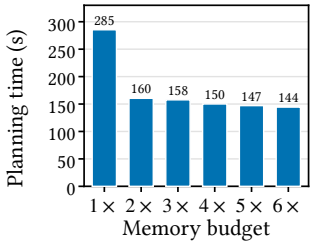


Figure 7: Planning scalability.

EFFVEDA and SQUID finish and SQUID attains the strongest measured high-recall performance. The result establishes policy independence for the tested role- and attribute-derived workloads, without claiming coverage of every authorization policy.

6.3 Memory Footprint

Table 3 compares realized memory and QPS at measured Recall@10 near 0.95 on SIFT+Tree-based. SQUID uses a 1.5× vector-copy budget, whereas the compared baselines are not constrained to this budget. SQUID reaches 3,545 QPS with 2,116 MB, the smallest footprint and highest QPS among the compared partitioning methods. HONEYBEE, EFFVEDA, and VEDA consume 2.00×, 1.88×, and 1.86× SQUID's memory while providing 0.78×, 0.91×, and 0.95× its QPS, respectively.

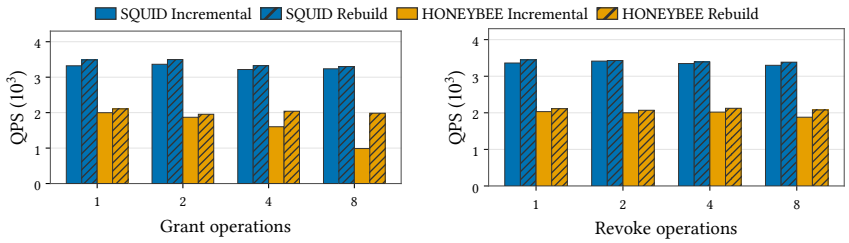
Figure 6 further reports realized footprint across vector-copy budgets. Because the planning budget excludes per-partition indexes and method-specific metadata, the footprint includes materialized vector relations, their ANN indexes, and persistent planning metadata but excludes common source tables and transient construction memory. The 1× reference is the 1.385 GB footprint of one indexed SIFT copy. The differences at equal copy budgets confirm that vector-copy count and realized database memory must be reported separately.

6.4 Planning and Maintenance

6.4.1 Partition Planning. We first measure the Partition Planner's initialization time on SIFT+ERBAC. Figure 7 reports the time from resolved visibility to a completed partition plan across 1×–6× vector-copy budgets, excluding partition materialization and ANN-index construction. Planning takes 285.4 seconds at 1× and 144.4 seconds at 6×, and every tested budget completes within five minutes. The Planner therefore produces every tested organization for this workload within the evaluation's construction limit.

6.4.2 Incremental Maintenance. We evaluate whether incremental maintenance preserves **query efficiency** relative to complete rebuilding. On SIFT+Tree-based, each batch contains $n \in \{1, 2, 4, 8\}$ grant or revoke operations. For each system, *incremental* updates the existing partition organization, whereas *rebuild* reconstructs it from the same post-update visibility. Both modes use the same query workload and recall target, so their QPS difference isolates the query-performance loss caused by incremental maintenance. Figure 8 reports the grant and revoke results separately.

For grants, SQUID's incremental QPS remains within 4.75% of complete rebuilding across all batch sizes. For revokes, the corresponding gap remains within 2.53%. Incremental maintenance sustains 3.2–3.4K QPS, and the gap from rebuilding does not widen as the number of operations increases. Affected-region repair therefore preserves most of the query efficiency recovered by complete reconstruction. HONEYBEE remains close to rebuilding only for small batches. At eight grants, its incremental QPS falls 50.25% below rebuilding, while at eight revokes it falls 9.51% below rebuilding. SQUID's incrementally maintained organization remains 1.58–1.72× faster than fully rebuilt HONEYBEE in every tested setting. Grants are more disruptive because they may require new route coverage and placements, whereas revokes primarily trigger cleanup. SQUID handles both update classes within the affected region without accumulating substantial query-performance loss.



(a) Grant operations. (b) Revoke operations.

Figure 8: Maintenance QPS after grant/revoke batches.

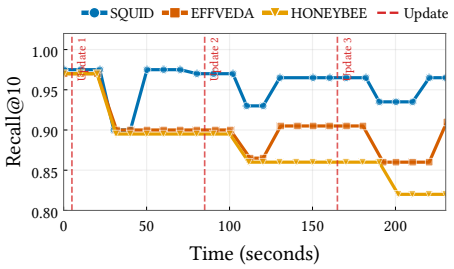


Figure 9: Recall during incremental maintenance.

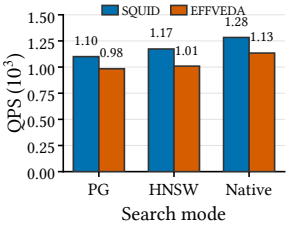


Figure 10: Search Engine execution modes.

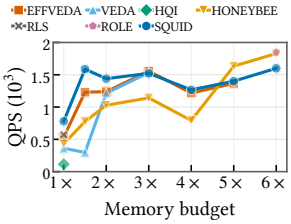


Figure 11: QPS at Recall@10 $\geq$ 95% across copy budgets.

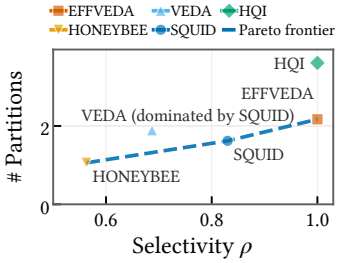


Table 5: Partition plan measurements.

Method	ρ	#P	GB
HONEYBEE	0.563	1.06	2.117
SQUID	**0.830**	**1.62**	**2.115**
VEDA	0.687	1.89	2.241
EFFVEDA	1.000	2.17	2.343
HQI	1.000	3.61	1.389

Figure 12: Route-quality Pareto frontier on SIFT+Tree-based. Higher ρ and lower #P are preferable. The dashed line connects the observed Pareto-optimal methods.

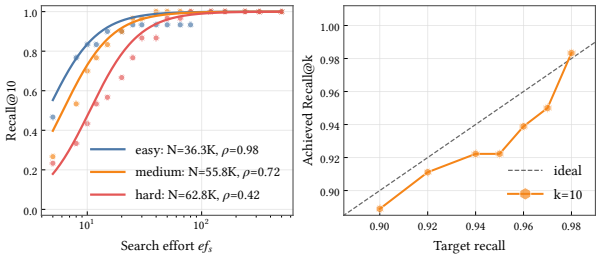


Figure 13: Recall-model fit (left) and achieved recall at the predicted search depth (right).

Figure 9 evaluates **recall recovery** while foreground queries execute continuously. We replay a fixed panel of 50 update-affected queries every 10 seconds and report their mean Recall@k. Three batches of 100 updates begin at approximately 5, 85, and 165 seconds. All methods use HNSW with method-specific search parameters tuned to their steady-state recall.

SQUID starts near 0.975 recall and falls to approximately 0.900, 0.930, and 0.935 after the three update batches. After each drop, recall returns to 0.965–0.975 as SQUID repairs and publishes the affected routes while structural maintenance continues in the background. EFFVEDA falls to approximately 0.90 after the first batch and to roughly 0.86 after the second; its later recovery coincides with the completion of background reconstruction. HONEYBEE does not complete its first reconstruction within the 240-second observation window. Its successive update effects therefore accumulate, and recall declines to approximately 0.82 after the third batch. These results show that rapid route repair limits SQUID's recall-degradation window, whereas reconstruction-based maintenance can suffer persistent loss when rebuilding cannot keep pace with updates.

6.5 System Mechanisms and Sensitivity

6.5.1 Partition-Plan Quality. We evaluate the route quality of partition plan produced on SIFT+Tree-based under an approximate 1.5× vector-copy budget. For each plan, ρ denotes mean routing selectivity over routed user–partition pairs, and #P denotes the average number of partitions on a user route.

Figure 12 places SQUID at the knee of the observed Pareto frontier, with routing selectivity $\rho = 0.830$ and mean fan-out #$P = 1.62$. Compared with HONEYBEE, SQUID improves selectivity by 47% at the cost of only 0.56 additional routed partitions, while their footprints remain nearly identical (2.115 versus 2.117 GB). SQUID also strictly dominates VEDA, which achieves lower selectivity (0.687), higher fan-out (1.89), and a larger footprint (2.241 GB). EFFVEDA and HQI reach filtering-free routes with $\rho = 1.000$, but increase fan-out to 2.17 and 3.61, respectively. In particular, compared with EFFVEDA, SQUID reduces fan-out by 0.55 partitions and storage by 0.228 GB while retaining high selectivity. Although HQI uses only 1.389 GB, its fan-out is more than twice that of SQUID. SQUID provides the most favorable balance of routing selectivity, query fan-out, and storage under the constrained vector-copy budget.

6.5.2 Query-Execution Benefit. We isolate query execution under a 1.5× budget on SIFT+ERBAC by holding each method's plan and query stream fixed. Figure 10 compares PostgreSQL HNSW compensation, direct HNSW execution, and each method's native execution mode. SQUID reaches 1,283 QPS, exceeding direct HNSW by 9.4%, PostgreSQL compensation by 16.6%, and EFFVEDA's native mode by 13.1%. This ablation measures the aggregate benefit of SQUID's execution path but does not separately attribute the gain to route ordering and adaptive effort allocation.

6.5.3 Recall-Aware Cost Calibration. We fit the recall model by weighted nonlinear least squares over measured recall, partition

size, route selectivity, k, and ef_s. Figure 13 compares fitted and measured Recall@10 curves and evaluates the search depths predicted for target recall. The fitted curves reproduce the measured saturation trend and preserve the ordering of the three route settings. Achieved recall differs from the target by at most approximately 0.03, although the model undershoots between 0.92 and 0.97. The model is therefore suitable for approximate plan ranking but not as a recall guarantee, so all query results use measured recall after independent search-depth sweeps.

6.5.4 Candidate-Graph Sparsity. We vary $d \in \{4, 8, 16, 32, 64\}$ on SIFT+Tree-based while holding the workload and memory setting fixed. Table 4 shows planning-stage times of 2.020–2.188 seconds and query times of 3.15–4.95 ms. The lowest degree $d = 4$ reaches 3.663 ms with the shortest planning time, but query time is non-monotonic and does not identify a query-optimal degree. We use $d = 32$ to retain more candidates at only 0.063 seconds, or 3.1%, more planning time than $d = 4$. The bounded ranges indicate limited sensitivity over the tested degrees.

6.5.5 Memory-Budget Sensitivity. We plan SIFT+ERBAC from 1× to 6× and select the smallest search depth reaching Recall@10 near 0.95 for each method and budget. Figure 11 shows that SQUID produces a plan at every budget, with QPS ranging from 780 to 1,600. At 1.5×, SQUID reaches 1,588 QPS, 29% above EFFVEDA and 2.05× HONEYBEE. SQUID does not dominate at the largest budgets, where HONEYBEE and unconstrained ROLE reach higher QPS. Its advantage is therefore concentrated under tight memory budgets.

6.5.6 Result-Size Sensitivity. We reuse one partition plan for $k \in \{1, 10, 50, 100\}$ and independently sweep search depth for each result size. Figure 14 reports the resulting QPS–Recall@k curves. Near 0.95 recall, $k = 1$ reaches approximately 3.1K QPS, while $k = 10$, 50, and 100 reach 2.5–2.9K QPS. The limited spread among the multi-result settings shows that high-recall throughput is moderately sensitive to k over the tested range.

6.5.7 Compatibility with Filtered ANN Indexes. We test whether SQUID remains useful when its mixed partitions use ACORN rather than HNSW. On SIFT+Tree-based, SQUID uses a 1.2× copy budget, ACORN for mixed partitions, and HNSW for pure partitions without refitting its HNSW-derived cost model. Figure 15 compares SQUID+ACORN with one collection-wide ACORN index and fully materialized ROLE partitions. SQUID+ACORN exceeds collection-wide ACORN throughout their common recall range. Near 0.95 recall, SQUID+ACORN reaches about 3.0K QPS, roughly 6× collection-wide ACORN's throughput; the ratio remains about 5× near 0.98 recall. ROLE is faster over part of the high-recall range but requires roughly 3.5× vector copies, compared with SQUID+ACORN's 1.2× budget. This single-workload result establishes compatibility rather than index-independent optimality: partitioning remains beneficial when ACORN replaces HNSW in mixed partitions, even though the recall-aware cost model is not refitted for ACORN.

7 RELATED WORK

General ANN indexing and systems. ANN search has been studied through locality-sensitive hashing [2, 6], vector quantization [4, 11], and graph navigation [24, 25]. Database systems such as

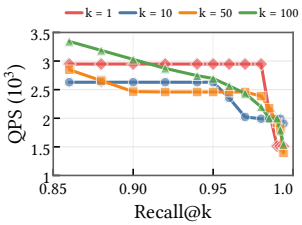


Figure 14: QPS–Recall@k across result sizes.

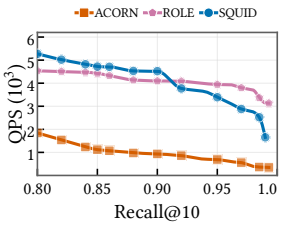


Figure 15: ACORN compatibility on SIFT+Tree-based.

PASE [42] and mixed vector–relational access paths [29] integrate vector search with relational execution. These methods optimize search within a fixed physical collection, whereas SQUID determines visibility-derived search spaces and can use an ANN index inside each partition.

Filtered-index methods. Filtered ANNS systems attach predicates to a shared index rather than materializing a physical layout for each query scope [12, 27, 38, 41, 45]. ACORN [27] and Filtered-DiskANN [12] modify graph traversal, while SeRF [45] and SIEVE [22] build or select predicate-aware structures. HQANN [40] similarly combines vector search with structured predicates. These methods remain effective substrates for SQUID, but a collection-wide filtered index can still examine many invisible vectors when a user's resolved visible set is sparse.

Partition-based methods. Partition-based systems expose searchable physical units and route queries only to units that may contain valid results [13, 16, 26, 44]. HQI [26], Curator [16], HONEYBEE [44], and VEDA [13] exploit roles, groups, labels, or visible-set relations and use replication to trade memory for lower query computation. Their planning units are commonly derived from a stable authorization structure, so policy evolution may require reconstruction or produce coarse mixed partitions. SQUID instead derives Access Atoms from resolved visibility and jointly decides copy retention, common-recall route cost, and local repair without assuming one policy representation.

8 CONCLUSION

Access-controlled vector search requires physical organization over resolved visibility rather than only query-time filtering or partitions derived from a particular authorization policy. SQUID represents visibility-equivalent vectors as Access Atoms and uses sparse Shared-Atom candidate generation with recall-aware ranking to retain beneficial copies under a memory budget. Its route-aware Search Engine coordinates search over the resulting mixed partitions, while the Partition Maintainer enforces current visibility and repairs affected copies, routes, indexes, and metadata without routine global reconstruction. Across RBAC- and ABAC-derived workloads, SQUID improves throughput at matched recall under constrained replication, remains compatible with ACORN, and localizes maintenance after vector and visibility updates.

REFERENCES

[1] Muhammad Umar Aftab, Ali Hamza, Ariyo Oluwasanmi, Xuyun Nie, Muhammad Shahzad Sarfraz, Danish Shehzad, Zhiguang Qin, and Ammar Rafiq. 2022. Traditional and Hybrid Access Control Models: A Detailed Survey. *Security and Communication Networks* 2022 (2022), 1–12. https://doi.org/10.1155/2022/1560885

[2] Alexandr Andoni and Piotr Indyk. 2006. Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor in High Dimensions. In *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*. 459–468. https://doi.org/10.1109/focs.2006.49

[3] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2017. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. In *Lecture Notes in Computer Science*. 34–49. https://doi.org/10.1007/978-3-319-68474-1_3

[4] Artem Babenko and Victor Lempitsky. 2014. Additive Quantization for Extreme Vector Compression. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*. 931–938. https://doi.org/10.1109/cvpr.2014.124

[5] Gunjan Batra. 2024. Attribute-Based Access Control. In *Encyclopedia of Cryptography, Security and Privacy*. 1–3. https://doi.org/10.1007/978-3-642-27739-9_1539-1

[6] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*. 253–262. https://doi.org/10.1145/997817.997857

[7] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. https://doi.org/10.1109/tbdata.2025.3618474

[8] David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. 2001. Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security* 4, 3 (2001), 224–274. https://doi.org/10.1145/501978.501980

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474. https://doi.org/10.14778/3303753.3303754

[10] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27. https://doi.org/10.1145/3589282

[11] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. https://doi.org/10.1109/tpami.2013.240

[12] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416. https://doi.org/10.1145/3543507.3583552

[13] Shanshan Han, Vishal Chakraborty, and Sharad Mehrotra. 2026. Don't Stir the Pot! Authorized Vector Data Retrieval via Access-Aware Indexing. https://doi.org/10.48550/arXiv.2605.01342 arXiv:2605.01342 [cs.DB]

[14] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Technical Report. National Institute of Standards and Technology. https://doi.org/10.6028/nist.sp.800-162

[15] H Jégou, M Douze, and C Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. https://doi.org/10.1109/tpami.2010.57

[16] Yicheng Jin, Yongji Wu, Wenjun Hu, Bruce M. Maggs, Xiao Zhang, and Danyang Zhuo. 2024. Curator: Efficient Indexing for Multi-Tenant Vector Databases. *arXiv preprint arXiv:2401.07119* (2024). https://doi.org/10.48550/arXiv.2401.07119

[17] Jeff Johnson, Matthijs Douze, and Herve Jegou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. https://doi.org/10.1109/tbdata.2019.2921572

[18] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550

[19] Axel Kern, Andreas Schaad, and Jonathan Moffett. 2003. An administration concept for the enterprise role-based access control model. In *Proceedings of the eighth ACM symposium on Access control models and technologies*. 3–11. https://doi.org/10.1145/775412.775414

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html

[21] Ninghui Li, Tiancheng Li, Ian Molloy, Qihua Wang, Elisa Bertino, Seraphin Calo, and Jorge Lobo. 2007. *Role Mining for Engineering and Optimizing Role Based Access Control Systems*. Technical Report CERIAS TR 2007-60. Center for Education and Research in Information Assurance and Security (CERIAS), Purdue University. https://www.cerias.purdue.edu/apps/reports_and_papers/view/3329

[22] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4723–4736. https://doi.org/10.14778/3749646.3749725

[23] David G. Lowe. 2004. Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision* 60, 2 (2004), 91–110. https://doi.org/10.1023/b:visi.0000029664.99615.94

[24] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68. https://doi.org/10.1016/j.is.2013.10.006

[25] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. https://doi.org/10.1109/tpami.2018.2889473

[26] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-Throughput Vector Similarity Search in Knowledge Graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25. https://doi.org/10.1145/3589777

[27] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27. https://doi.org/10.1145/3654923

[28] PostgreSQL Global Development Group. 2026. *PostgreSQL 17 Documentation*. https://www.postgresql.org/docs/17/ Accessed: 2026-07-15.

[29] Viktor Sanca and Anastasia Ailamaki. 2024. Efficient Data Access Paths for Mixed Vector-Relational Search. In *Proceedings of the 20th International Workshop on Data Management on New Hardware*. 1–9. https://doi.org/10.1145/3662010.3663448

[30] R.S. Sandhu, E.J. Coyne, H.L. Feinstein, and C.E. Youman. 1996. Role-based access control models. *Computer* 29, 2 (1996), 38–47. https://doi.org/10.1109/2.485845

[31] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, Dmitry Baranchuk, Edo Liberty, Frank Liu, Ben Landrum, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Yizhuo Chen, Weiguo Zheng, Zihao Wan, Jie Yin, and Ben Huang. 2024. Results of the Big ANN: NeurIPS'23 competition. https://doi.org/10.48550/arXiv.2409.17424 arXiv:2409.17424 [cs.IR]

[32] Liwen Sun, Michael J. Franklin, Sanjay Krishnan, and Reynold S. Xin. 2014. Fine-grained partitioning for aggressive data skipping. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. 1115–1126. https://doi.org/10.1145/2588555.2610515

[33] Liwen Sun, Michael J. Franklin, Jiannan Wang, and Eugene Wu. 2016. Skipping-oriented partitioning for columnar layouts. *Proceedings of the VLDB Endowment* 10, 4 (2016), 421–432. https://doi.org/10.14778/3025111.3025123

[34] Bart Thomee, David A. Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. 2016. YFCC100M. *Commun. ACM* 59, 2 (2016), 64–73. https://doi.org/10.1145/2812802

[35] Timescale. 2024. wikipedia-22-12-simple-embeddings. https://huggingface.co/datasets/timescale/wikipedia-22-12-simple-embeddings Dataset revision 0a929ebdbfdd9d7a87d61450d9154575c67cbda6; accessed 2026-07-15.

[36] Jaideep Vaidya, Vijayalakshmi Atluri, and Janice Warner. 2006. RoleMiner. In *Proceedings of the 13th ACM conference on Computer and communications security*. 144–153. https://doi.org/10.1145/1180405.1180424

[37] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627. https://doi.org/10.1145/3448016.3457550

[38] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2023. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint. In *Advances in Neural Information Processing Systems 36*. 15738–15751. https://doi.org/10.52202/075280-0692

[39] James N. Weiss. 1997. The Hill Equation Revisited: Uses and Misuses. *The FASEB Journal* 11, 11 (1997), 835–841. https://doi.org/10.1096/fasebj.11.11.9285481

[40] Wei Wu, Junlin He, Yu Qiao, Guoheng Fu, Li Liu, and Jin Yu. 2022. HQANN: Efficient and Robust Similarity Search for Hybrid Queries with Structured and Unstructured Constraints. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 4580–4584. https://doi.org/10.1145/3511808.3557610

[41] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26. https://doi.org/10.1145/3698814

[42] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *Proceedings of*

the 2020 ACM SIGMOD International Conference on Management of Data. 2241–2253. https://doi.org/10.1145/3318464.3386131

[43] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. 2020. Qd-tree: Learning Data Layouts for Big Data Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 193–208. https://doi.org/10.1145/3318464.3389770

[44] Hongbin Zhong, Matthew Lentz, Nina Narodytska, Adriana Szekeres, and Kexin Rong. 2026. HONEYBEE: Efficient Role-based Access Control for Vector Databases via Dynamic Partitioning. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–26. https://doi.org/10.1145/3786625

[45] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26. https://doi.org/10.1145/3639324

A ALIGNING PLANNING AND QUERY EXECUTION

SQUID uses different heuristics for offline partition planning and query execution because the two stages operate under different latency and information constraints. Before indexes are materialized, the Partition Planner must compare many candidate partition plans without executing the future query workload for each plan. It therefore uses the recall-aware cost model in §4.4 as an offline surrogate for expected query computation at a common recall target. At query time, applying and inverting this model for every routed partition would add computation to the latency-critical path, and its static inputs cannot capture the candidate distribution of the current query. The Search Engine therefore uses the selectivity-guided probing policy in §5.1 to allocate search depth from query-local observations.

A.1 Detailed Search Procedure

For a query $Q = (q, u, k)$, the User Routing Directory returns the routed partitions $\mathcal{P}(u)$ and their route selectivities $\rho(u, P) = |D_u \cap P|/|P|$. The Search Engine visits these partitions in decreasing route selectivity so that high-selectivity routes can establish a competitive global result early. It maintains an authorized candidate set R, where $\delta_k(R)$ is the distance of the current k-th candidate if $|R| \geq k$ and $+\infty$ otherwise. For each routed partition P, an initial search with depth ef_0 returns candidates $C_0(P)$ and the authorized subset

$$A_0(P) = \{x \in C_0(P) \mid x \in D_u\}.$$

If $A_0(P)$ contains a candidate closer than $\delta_k(R)$, SQUID estimates the query-local authorized ratio

$$\rho_0(P) = \frac{|A_0(P)|}{|C_0(P)|}$$

and expands the search to

$$ef_P = \left\lceil \frac{ef_0}{\rho_0(P)} \right\rceil.$$

If $A_0(P) \neq \emptyset$ but none of its candidates can improve the current result, SQUID does not expand that partition. An empty $A_0(P)$, however, is inconclusive because the initial search may miss sparse authorized vectors. SQUID therefore falls back to the cataloged route selectivity and uses

$$ef_P = \left\lceil \frac{ef_0}{\rho(u, P)} \right\rceil.$$

After each expansion, the Search Engine filters unauthorized candidates, merges authorized candidates into R, and updates $\delta_k(R)$. The tighter threshold makes later expansion decisions depend on the results already obtained from earlier routed partitions. Algorithm 3 gives the corresponding control flow.

A.2 Heuristic Alignment

Using two heuristics raises a compositional question. The Partition Planner selects a partition plan using its offline estimate; after materialization, the resulting partition organization is served by the Search Engine's selectivity-guided probing policy. The relevant requirement is therefore not that the two heuristics produce identical search depths. Instead, the offline estimate should preserve the ordering of the expected query computation that the selectivity-guided probing policy achieves on candidate partition plans. This section gives sufficient conditions under which the two-stage design satisfies this requirement.

Scope. Fix a workload distribution, target recall τ, and selectivity-guided probing policy π. For a feasible partition plan $\mathcal{P}$, let $C^\pi(\mathcal{P})$ denote the expected query computation obtained after materializing the plan and having the Search Engine execute $\mathcal{P}$ using π under this setting. Let $\widehat{C}(\mathcal{P})$ denote the offline planning estimate, corresponding to $C(\mathcal{P})$ in §4.4. The analysis assumes a bounded difference between $\widehat{C}(\mathcal{P})$ and $C^\pi(\mathcal{P})$ over the candidate partition plans being compared. It establishes the resulting composition and ordering guarantees, but it does not establish that the fitted model has such a uniform bound for every workload or recall target. It also does not guarantee the recall of an individual query or directly order wall-clock latency.

At one step of the Partition Planner, let $\mathfrak{C}$ be the finite set of candidate partition plans produced by the currently valid local consolidation operations. Define the candidate selected by the recall-aware cost model and the best candidate under the selectivity-guided probing policy as

$$\widehat{\mathcal{P}} \in \arg\min_{\mathcal{P} \in \mathfrak{C}} \widehat{C}(\mathcal{P}), \qquad \mathcal{P}^\star_\pi \in \arg\min_{\mathcal{P} \in \mathfrak{C}} C^\pi(\mathcal{P}).$$

Proposition A.1 (Composition of partition planning and query execution). *Suppose that*

$$\left|\widehat{C}(\mathcal{P}) - C^\pi(\mathcal{P})\right| \leq \epsilon$$

for every $\mathcal{P} \in \mathfrak{C}$. Then

$$C^\pi(\widehat{\mathcal{P}}) \leq C^\pi(\mathcal{P}^\star_\pi) + 2\epsilon.$$

Proof. The error bound and the definition of $\widehat{\mathcal{P}}$ give

$$C^\pi(\widehat{\mathcal{P}}) \leq \widehat{C}(\widehat{\mathcal{P}}) + \epsilon \leq \widehat{C}(\mathcal{P}^\star_\pi) + \epsilon \leq C^\pi(\mathcal{P}^\star_\pi) + 2\epsilon.$$

□

Proposition A.1 formalizes why the two heuristics can be used together. The Partition Planner need not reproduce the per-query decisions of π; it only needs to estimate their expected consequence well enough to select the partition plan. Under the stated error bound, the candidate selected by the recall-aware cost model incurs at most 2ϵ more expected query computation than the best candidate available at that planning step when both use the same selectivity-guided probing policy. We next give pairwise and consolidation-operation-level forms of this result.

Proposition A.2 (Robust partition-organization ordering). *Let $\mathcal{P}_1$ and $\mathcal{P}_2$ be two feasible partition plans. Suppose that the planning estimator has absolute error at most ϵ on both candidates:*

$$\left|\widehat{C}(\mathcal{P}_i) - C^\pi(\mathcal{P}_i)\right| \leq \epsilon, \qquad i \in \{1, 2\}.$$

If

$$\widehat{C}(\mathcal{P}_1) + 2\epsilon < \widehat{C}(\mathcal{P}_2),$$

then

$$C^\pi(\mathcal{P}_1) < C^\pi(\mathcal{P}_2).$$

Proof. The assumed error bound gives

$$C^\pi(\mathcal{P}_1) \le \widehat{C}(\mathcal{P}_1) + \epsilon$$

and

$$C^\pi(\mathcal{P}_2) \ge \widehat{C}(\mathcal{P}_2) - \epsilon.$$

The separation condition implies

$$\widehat{C}(\mathcal{P}_1) + \epsilon < \widehat{C}(\mathcal{P}_2) - \epsilon.$$

Combining these inequalities yields

$$C^\pi(\mathcal{P}_1) \le \widehat{C}(\mathcal{P}_1) + \epsilon < \widehat{C}(\mathcal{P}_2) - \epsilon \le C^\pi(\mathcal{P}_2).$$

Therefore, $C^\pi(\mathcal{P}_1) < C^\pi(\mathcal{P}_2)$. □

Proposition A.2 shows that exact prediction is unnecessary for comparing partition plans. An estimation error can reverse the ordering only when the estimated costs of two candidates differ by at most 2ϵ. Consequently, decisions with a sufficiently large estimated cost margin remain robust even when the model does not accurately predict the search depth required by every individual query.

The Partition Planner applies this reasoning to local consolidation operations. For an operation o that changes the current partition plan $\mathcal{P}$ into $\mathcal{P}_o$, define the executed and estimated changes in query computation as

$$\Delta C_o^\pi = C^\pi(\mathcal{P}_o) - C^\pi(\mathcal{P})$$

and

$$\widehat{\Delta C}_o = \widehat{C}(\mathcal{P}_o) - \widehat{C}(\mathcal{P}).$$

Corollary A.3 (Robust consolidation-operation classification). *Suppose that*

$$\left|\widehat{C}(\mathcal{P}') - C^\pi(\mathcal{P}')\right| \le \epsilon$$

for the current partition plan $\mathcal{P}$ and every candidate partition plan $\mathcal{P}'$. Then

$$\left|\widehat{\Delta C}_o - \Delta C_o^\pi\right| \le 2\epsilon.$$

In particular, if $\widehat{\Delta C}_o < -2\epsilon$, then $\Delta C_o^\pi < 0$, and operation o reduces the expected query computation under π. Likewise, if $\widehat{\Delta C}_o > 2\epsilon$, then $\Delta C_o^\pi > 0$.

Proof. By the triangle inequality,

$$\begin{aligned}\left|\widehat{\Delta C}_o - \Delta C_o^\pi\right| &= \left|\widehat{C}(\mathcal{P}_o) - C^\pi(\mathcal{P}_o) - \left(\widehat{C}(\mathcal{P}) - C^\pi(\mathcal{P})\right)\right| \\ &\le \left|\widehat{C}(\mathcal{P}_o) - C^\pi(\mathcal{P}_o)\right| + \left|\widehat{C}(\mathcal{P}) - C^\pi(\mathcal{P})\right| \\ &\le 2\epsilon.\end{aligned}$$

If $\widehat{\Delta C}_o < -2\epsilon$, then

$$\Delta C_o^\pi \le \widehat{\Delta C}_o + 2\epsilon < 0.$$

The case $\widehat{\Delta C}_o > 2\epsilon$ follows symmetrically. □

When multiple memory-reducing operations remain, SQUID ranks them by the estimated increase in query computation per unit of saved memory. Let $\Delta M_o > 0$ denote the memory released by operation o, and define

$$r_o^\pi = \frac{\Delta C_o^\pi}{\Delta M_o}, \qquad \widehat{r}_o = \frac{\widehat{\Delta C}_o}{\Delta M_o}.$$

Memory consumption is computed directly from the materialized copies, so ΔM_o is not estimated by the recall-aware cost model.

Corollary A.4 (Robust ordering of memory-release operations). *For two candidate operations o_1 and o_2, suppose the query-computation estimation error is bounded by ϵ for the current and resulting partition plans. If*

$$\widehat{r}_{o_1} + \frac{2\epsilon}{\Delta M_{o_1}} + \frac{2\epsilon}{\Delta M_{o_2}} < \widehat{r}_{o_2},$$

then

$$r_{o_1}^\pi < r_{o_2}^\pi.$$

Proof. Corollary A.3 implies

$$\left|\widehat{r}_o - r_o^\pi\right| = \frac{\left|\widehat{\Delta C}_o - \Delta C_o^\pi\right|}{\Delta M_o} \le \frac{2\epsilon}{\Delta M_o}.$$

Therefore,

$$r_{o_1}^\pi \le \widehat{r}_{o_1} + \frac{2\epsilon}{\Delta M_{o_1}} < \widehat{r}_{o_2} - \frac{2\epsilon}{\Delta M_{o_2}} \le r_{o_2}^\pi.$$

Hence, $r_{o_1}^\pi < r_{o_2}^\pi$. □

Interpretation and limitations. The results above explain why exact cost prediction is unnecessary for robust planning decisions, but they remain conditional on the assumed error bound. They do not show that the fitted recall-aware cost model has a uniform ϵ for every workload, partition plan, or recall target. The bound may be particularly loose at high recall targets, where a small recall-prediction error can induce a large error after the saturation curve is inverted. SQUID consequently uses the recall-aware cost model to compare expected query computation during offline partition planning rather than to provide a query-time recall guarantee.

The definition of C^π fixes the workload distribution, recall target, and selectivity-guided probing policy. The propositions concern query computation and do not directly guarantee wall-clock latency. The composition guarantee is local to each candidate-selection step and does not turn SQUID's greedy consolidation sequence into a globally optimal solution of MBP^2. They also leave candidates inside the uncertainty margin unordered: when the estimated query computation of two partition plans differ by at most 2ϵ, either ordering may be consistent with the assumed error bound. In this regime, SQUID may retain the current partition plan or apply a deterministic secondary criterion, such as greater memory reduction, instead of treating an uncertain cost difference as conclusive.

B RECALL-MODEL TRAINING

This section describes how SQUID calibrates the recall model in §4.4. The calibration is an offline profiling procedure for the deployed ANN index configuration; it is not performed while planning or serving queries. Its goal is to estimate how the HNSW search depth required for Recall@k changes with partition size and route selectivity.

Profiling samples. We construct representative vector collections at several cardinalities N, build an HNSW index for each collection using the same index family and construction parameters as the deployment, and select a fixed set of query vectors. For each collection, we generate controlled visibility predicates that retain

fractions ρ of the vectors, spanning clean routes ($\rho = 1$) and increasingly mixed routes. We then sweep a grid of ef_s values. Thus, each profiling point is identified by (N, ρ, k, ef_s), rather than by a particular authorization policy or a particular partition plan.

For every query and profiling point, an exact filtered top-k search gives the authorized ground-truth result set G. The HNSW search under the same visibility predicate returns A, and the measured response is

$$\mathrm{Rec}@k = \frac{|A \cap G|}{|G|}. \tag{12}$$

We average this quantity over the profiling queries. The profiler retains a measurement only when the database executes the approximate query through the HNSW index; samples for which the optimizer switches to a different execution strategy are outside the scope of this HNSW recall model.

Profiling protocol. The profiler constructs each collection deterministically from the source vectors and derives nested synthetic visibility predicates by assigning every vector a stable hash bucket. A predicate retaining fraction ρ keeps a fixed prefix of these buckets, so the samples at different selectivities share the same vectors and queries rather than confounding selectivity with a new collection draw. For every sampled N, the same query panel is evaluated for every pair (ρ, ef_s). Exact evaluation disables ANN access paths, whereas approximate evaluation enables HNSW and sets its search depth to ef_s. The HNSW indexes use the same $M = 16$ and $ef_{\text{construction}} = 64$ configuration as the experiments. The profiler records the execution-plan class together with every measurement and excludes any point that is not served by HNSW from the Hill fit. This makes the fitted curve a model of HNSW behavior, rather than of optimizer decisions such as filtering first and sorting afterward.

Two-stage Hill fit. For every profiling point, we first normalize search effort by the expected number of visible candidates per requested result, $x = ef_s\rho/k$. For each sampled cardinality N, we fit the Hill curve

$$\mathrm{Rec}(x; N) = \frac{x^{\gamma_N}}{\lambda_N^{\gamma_N} + x^{\gamma_N}} \tag{13}$$

to its HNSW measurements by bounded nonlinear least squares. The per-size fits estimate both the saturation exponent γ_N and the effort scale λ_N. SQUID uses the median of the fitted γ_N values as a single global exponent γ, refits one λ_N for each sampled size with γ fixed, and finally fits $\lambda_N = \lambda_0 + \lambda_1 \log(1 + N)$ by least squares. This produces the three parameters $\gamma, \lambda_0, \lambda_1$ used in the recall equation in §4.4.

Using the fitted model. Given a planning target τ, SQUID first inverts the fitted curve for a clean route to obtain its base search depth,

$$ef_s^{\text{base}}(N, k, \tau) = k\lambda(N)\left(\frac{\tau}{1-\tau}\right)^{1/\gamma}. \tag{14}$$

For a route with selectivity $\rho(u, P)$, it uses $ef_s^{\text{base}}/\rho(u, P)$, clipped to the feasible range for the partition, as the recall-normalized search-depth estimate. Hence the model does not assume that a fixed ef_s gives comparable recall across layouts: a larger partition changes $\lambda(N)$, while a mixed route decreases the effective visible-candidate budget through $\rho(u, P)$. If the ANN index configuration, vector distribution, or target k changes materially, the same profiling procedure is rerun to recalibrate the model.

Latency-surrogate calibration. SQUID fits the latency surrogate separately from the recall curve. It reuses the sampled partition sizes and selectivities, evaluates several base search depths, and converts each to a route depth by $\lceil ef_s^{\text{base}}/\rho \rceil$, subject to the partition-size cap. For HNSW-only measurements, it fits the nonnegative coefficients in

$$T(N, ef_s, \rho) = a\log(1 + N) + c_0 + b\log(1 + N)\left\lceil \frac{ef_s^{\text{base}}}{\rho} \right\rceil.$$

The profile uses repeated executions and records a robust per-setting latency summary before fitting; a deterministic held-out subset is used only to check calibration quality. The fitted surrogate is used to rank planning candidates, not to predict the latency of an individual query. Accordingly, measurements whose execution plan is not HNSW are excluded from this fit as well, and the online evaluation independently sweeps search depth and reports measured Recall@k.

C PARTITION-PLANNING DETAILS

This appendix supplies the formal details omitted from Section 4. They justify the seed accounting, candidate sparsification, and coverage-preserving local updates used by the Planner; the main text retains the corresponding method definitions and control flow.

C.1 Decision Version and Complexity

The decision version of MBP^2 takes an Access Atom Catalog, a memory budget $\mathcal{B}$, a query-computation threshold $\mathcal{K}$, and the per-route costs $C(u, P)$. It asks whether there exists a partition plan $\mathcal{P}$ and routes $\{\mathcal{P}(u)\}_{u \in U}$ satisfying route coverage,

$$D_u \subseteq \bigcup_{P \in \mathcal{P}(u)} P \quad \text{for every } u \in U,$$

the copy budget $M(\mathcal{P}) \le \mathcal{B}$, and $C(\mathcal{P}) \le \mathcal{K}$. The following theorem establishes the hardness cited in Section 4.1.

Theorem C.1. MBP^2 *is NP-hard.*

Proof. Consider a restricted family of MBP^2 instances with n independent gadgets. Gadget i is parameterized by positive integers v_i and q_i and contains two users a_i, b_i. Their valid search domains consist of a shared Access Atom S_i of cardinality v_i and respective singleton private atoms X_i and Y_i. This planning instance admits the two-partition plan

$$P_{a_i} = X_i \cup S_i, \qquad P_{b_i} = Y_i \cup S_i. \tag{15}$$

The construction specifies a valid instance of the general planning problem; it does not require SQUID's seed-generation procedure. The reduction uses the abstract per-route cost supplied with an MBP^2 instance. For $u \in \{a_i, b_i\}$, set

$$C(u, P) = \frac{\alpha_i}{\rho(u, P)}, \qquad \alpha_i = \frac{q_i(v_i + 1)}{2}. \tag{16}$$

Thus, q_i is encoded in the route-cost table, rather than in a user or workload weight. Different item gadgets share neither users nor atoms.

Within gadget i, retaining two copies of S_i incurs no consolidation penalty, whereas materializing one copy saves exactly v_i vectors. The least expensive one-copy organization places X_i, S_i, and Y_i together. Each user then sees $v_i + 1$ of its $v_i + 2$ vectors, so consolidation increases query computation by

$$2\alpha_i \left(\frac{v_i + 2}{v_i + 1} - 1 \right) = q_i. \tag{17}$$

Absorbing S_i into either private side increases computation by $\alpha_i(v_i + 1)/v_i \geq q_i$, and isolating S_i increases it by $2\alpha_i \geq q_i$; the remaining reorganization is no cheaper. Thus, every other one-copy organization for this gadget has query-computation increase at least q_i.

Let $x_i \in \{0, 1\}$ indicate whether gadget i uses one shared copy, and let M_0 and C_0 denote the memory consumption and query computation of the two-copy organization. A plan satisfying memory budget $\mathcal{B}$ and decision threshold $\mathcal{K}$ must therefore satisfy

$$\sum_{i=1}^{n} v_i x_i \geq M_0 - \mathcal{B}, \qquad \sum_{i=1}^{n} q_i x_i \leq \mathcal{K} - C_0. \tag{18}$$

It remains to show that the full planning space introduces no cheaper alternative. A partition mixing atoms from different gadgets cannot remove a copy beyond the copy removed within each gadget. Splitting every such partition along gadget boundaries preserves memory consumption and route coverage, and it weakly increases each route's selectivity; under $C(u, P) = \alpha_i/\rho(u, P)$, this transformation does not increase query computation. Thus, whenever a feasible plan exists, a gadget-separated feasible plan exists as well. The two inequalities are consequently the decision form of 0–1 Knapsack: saved memory is item value, additional query computation is item weight, $M_0 - \mathcal{B}$ is the target value, and $\mathcal{K} - C_0$ is the capacity. Setting $\mathcal{B} = M_0 - V$ and $\mathcal{K} = C_0 + W$ maps any Knapsack target V and capacity W to the constructed planning instance in polynomial time and preserves feasibility. Therefore, MBP^2 is NP-hard. □

C.2 Seed Copy Accounting

For each user u, the per-user seed materializes one partition $P_u^{(0)} = D_u$. Hence, every Access Atom S_{A_i} is copied once for every user in its visible-user set A_i, and the seed consumes

$$M(\mathcal{P}^{(0)}) = \sum_{A_i \in \mathcal{A}} |A_i|\,|S_{A_i}|.$$

Equivalently, summing the vectors in each user-visible domain gives

$$M(\mathcal{P}^{(0)}) = \sum_{u \in U} |D_u|.$$

The equality follows because each vector belongs to exactly one Access Atom and contributes to precisely the domains of the users in that Atom's visible-user set.

C.3 Sparse Shared-Atom Candidates

For an Atom S_{A_i} with $r = |\mathcal{I}(S_{A_i})|$ owner partitions, the complete owner clique contains $\binom{r}{2}$ edges. The core-star keeps only the $r - 1$ edges between core(S_{A_i}) and every other owner. Thus every non-core copy appears on one retained edge, while the atom-induced graph size decreases from $O(r^2)$ to $O(r)$. The omitted edges represent alternative pairwise consolidation orders, so this construction deliberately sparsifies rather than exhausts the candidate space.

When several Atoms induce the same partition pair, their witnesses are accumulated in $\Gamma(P_x, P_y)$. SQUID scores every raw core-star edge by $\phi(P_x, P_y)$, retains the top-d raw incident edges of each partition, and takes the union of these retained choices as the candidate graph. Thus the pruning bound applies to all core-star edges, rather than preserving a separate unbounded set of star edges. Ties are resolved deterministically by partition identifiers. The resulting graph is a sparse opportunity graph: it preserves a direct representative edge for a selected shared-copy witness, but does not claim to enumerate every pairwise consolidation order.

C.4 Coverage-Preserving Route Updates

Consider a valid local transformation of edge (P_x, P_y) with output partitions $\mathcal{T}_o(P_x, P_y)$. Let

$$\mathcal{T}_o(u) = \left\{ Q \in \mathcal{T}_o(P_x, P_y) \;\middle|\; \exists S_{A_i} \in \mathcal{S}(Q) : u \in A_i \right\}.$$

For every user whose route contains an endpoint, the Planner replaces its endpoint entries by the outputs in $\mathcal{T}_o(u)$:

$$\mathcal{P}_o(u) = \big(\mathcal{P}(u) \setminus \{P_x, P_y\}\big) \cup \mathcal{T}_o(u).$$

Routes that contain neither endpoint are unchanged. In particular, a user is not routed to an output merely because another user can access an Atom in that output.

Every transformation in Section 4.3 preserves the Atom union,

$$\bigcup_{Q \in \mathcal{T}_o(P_x, P_y)} \mathcal{S}(Q) = \mathcal{S}(P_x) \cup \mathcal{S}(P_y).$$

Thus, every Atom reachable through either old endpoint remains reachable through at least one selected output partition, proving route coverage after the replacement. The transformation changes placements only; it does not change any Atom's visible-user set. Therefore, coverage ensures physical reachability and the Search Engine's visibility filtering preserves authorization correctness, including when an output partition contains Atoms visible to different users.

C.5 Local Transformation Accounting

For a retained edge (P_x, P_y), let $\Omega = \mathcal{S}(P_x) \cap \mathcal{S}(P_y)$, $X = \mathcal{S}(P_x) \setminus \Omega$, and $Y = \mathcal{S}(P_y) \setminus \Omega$. These three Atom sets are pairwise disjoint. Let $\mathcal{T}_o(P_x, P_y)$ denote the nonempty output partitions of any transformation o in Section 4.3. Every such transformation satisfies

$$\bigcup_{Q \in \mathcal{T}_o(P_x, P_y)} \mathcal{S}(Q) = X \cup \Omega \cup Y \tag{19}$$

and materializes every Atom in Ω, X, and Y exactly once. Relative to the two input endpoints, the immediate memory release is therefore

$$\Delta M_o = \sum_{S_{A_i} \in \Omega} |S_{A_i}| > 0. \tag{20}$$

Only routes containing an endpoint can change. Let

$$U_e = \{u \in U \mid P_x \in \mathcal{P}(u) \;\vee\; P_y \in \mathcal{P}(u)\}. \tag{21}$$

After applying the coverage-preserving replacement rule, the corresponding query-computation change can be evaluated locally

as

$$\Delta C_o = \sum_{u \in U_e} \left(\sum_{Q \in \mathcal{P}_o(u) \cap \mathcal{T}_o(P_x, P_y)} C(u, Q) - \sum_{Q \in \mathcal{P}(u) \cap \{P_x, P_y\}} C(u, Q) \right). \tag{22}$$

All route contributions outside U_e cancel because their partitions and routes are unchanged. This locality permits the Planner to refresh only transformations incident to the newly materialized outputs.

C.6 Candidate Validity and Local Refresh

The candidate heap is maintained lazily. Each heap entry records its endpoint partition identifiers, the endpoint generations at which its five transformations were evaluated, and the selected operation's local $(\Delta C_o, \Delta M_o)$. Replacing, splitting, or retiring a partition increments its generation. A popped entry is valid only when both endpoints are still live and both stored generations equal the current generations; otherwise it is discarded without changing the plan. Accepted transformations invalidate cached entries incident to their retired endpoints, insert entries for the new outputs, and regenerate only the core-star edges induced by Atoms whose owner sets changed. If the heap contains no valid operation while the plan remains over budget, SQUID rebuilds the sparse graph and heap from the current Atom Inverted Table. These checks do not change the greedy objective; they ensure that every chosen operation is evaluated against the current local layout.

D INCREMENTAL-MAINTENANCE DETAILS

This section formalizes the invariants and transition rules maintained by the update procedure in Section 5.2.

D.1 Published-State Invariants

Let a published serving state be

$$\Sigma = (\mathcal{V}, \mathcal{A}, \mathcal{P}, \{\mathcal{P}(u)\}_{u \in U}, \mathcal{I}, \mathcal{G}), \tag{23}$$

where $\mathcal{A}$ is the Access Atom Catalog, $\mathcal{I}$ the Atom Inverted Table, and $\mathcal{G}$ the Shared-Atom Graph; per-partition indexes are associated with $\mathcal{P}$. A batch changes Σ to Σ'. SQUID publishes Σ' only after the following conditions hold:

$$d \in S'_{A_i} \iff \mathcal{V}'(d) = A_i, \qquad \forall d \in D', \tag{24}$$

$$D'_u \subseteq \bigcup_{P \in \mathcal{P}'(u)} P, \qquad \forall u \in U, \tag{25}$$

$$M(\mathcal{P}') \le \mathcal{B}. \tag{26}$$

Equation (24) gives Atom consistency, while Equations (25) and (26) give physical reachability and budget feasibility. Authorization safety is enforced against $\mathcal{V}'$ during candidate filtering, so every returned vector belongs to D'_u. New partitions and indexes are materialized before their routes become visible; obsolete copies are reclaimed only after the replacement state is published.

D.2 Batch Transition and Publication

SQUID separates logical visibility enforcement from physical reorganization. For a batch, it first applies the resolved-visibility change and uses the new relation $\mathcal{V}'$ for candidate filtering. Therefore a revoke or deletion is safe immediately, even if an old physical copy remains in a searchable index. Coverage Repair then reconstructs the affected Atoms, records tombstones for obsolete physical occurrences, and creates the copies required for every newly visible Atom. It identifies the directly touched partitions $\mathcal{P}_\Delta$, repairs their routes, and performs the budget and selectivity repair described below.

The resulting replacement partitions are built in a staging state together with their ANN indexes. SQUID publishes their partition metadata and route entries only after the indexes are ready and the invariants above hold. It then retires superseded routes and copies. Hence a grant becomes reachable when its new copy, index, and route are published; until that point, the previous published layout remains usable but does not claim to cover the newly granted vector. This publication order prevents a route from referring to an unmaterialized partition and makes the safety–reachability distinction explicit.

D.3 Local Repair Scope

Let $\mathcal{P}_\Delta$ contain partitions directly modified, created, or retired by a batch. The efficiency-repair scope is

$$\mathcal{P}^+_\Delta = \mathcal{P}_\Delta \cup \mathcal{N}_G(\mathcal{P}_\Delta). \tag{27}$$

The one-hop extension includes partitions whose consolidation opportunities can change because they share an Atom with the directly affected state. Candidate transformations are regenerated only for edges incident to this region. Partitions outside the region retain their Atom placements and indexes, and routes not covering a changed Atom retain their entries.

For an affected user, Route Repair first preserves every surviving route entry that still covers an Atom in its valid domain. If coverage is incomplete, it repeatedly adds the partition that covers the largest number of its uncovered Atoms; ties prefer higher route selectivity and then smaller partition size. The procedure stops once the user's visible Atom set is covered. It is applied only to users whose routes cover changed Atoms or whose visible Atom set changed, so unrelated route-directory entries remain stable.

D.4 Compaction and Split Semantics

For a partition P, let $L(P)$ and $T(P)$ be its live and inactive vector copies. Compaction is triggered when

$$\delta(P) = \frac{|T(P)|}{|L(P)| + |T(P)|} \ge \eta. \tag{28}$$

It replaces the physical contents and index of P with those induced by $L(P)$. Because it changes neither the live-Atom placements nor the route entries, compaction preserves Equations (24) and (25) while reducing physical storage.

A selectivity split replaces a live partition P by outputs P_1, P_2 satisfying

$$\mathcal{S}(P_1) \cup \mathcal{S}(P_2) = \mathcal{S}(P), \tag{29}$$

where $\mathcal{S}(P_1) \cap \mathcal{S}(P_2)$ contains only Atoms intentionally replicated for both output route groups. The route-replacement rule in Appendix C.4 preserves coverage. SQUID accepts the split only if its resulting organization $\mathcal{P}'$ satisfies $C(\mathcal{P}') < C(\mathcal{P})$ and $M(\mathcal{P}') \le \mathcal{B}$.

To choose a split, SQUID considers mixed partitions in the repair region in increasing average route selectivity, breaking ties by lower minimum selectivity and then larger partition size. It ranks routed

users by filtering impact $I(u,P) = |D_u \cap P|(1-\rho(u,P))$, forms seed groups from the least-overlapping high-impact pair, and assigns the remaining users in decreasing impact order according to their shared visible Atoms. An Atom visible to both output groups is replicated; every other Atom is assigned only to the group that needs it. The implementation caps the candidate seed set at 64 high-impact users before testing seed pairs. This cap bounds split-planning work without changing the coverage or budget acceptance tests.

E TIME-COMPLEXITY ANALYSIS

This appendix analyzes the logical work performed by SQUID. We distinguish it from physical ANN-index construction: the Planner determines Atom placements and routes, whereas materializing a final partition also invokes the chosen index implementation. Let $T_{\text{build}}(N)$ and $T_{\text{ann}}(N, ef)$ denote, respectively, the construction and query costs of the selected ANN index on N vectors at search depth ef. This makes the analysis applicable to HNSW as used in our implementation as well as to compatible per-partition indexes.

E.1 Notation

Let $n = |D|$, $m = |\mathcal{A}|$, and $p = |\mathcal{P}|$ denote the numbers of vectors, Access Atoms, and current partitions, respectively. The resolved-visibility relation contains

$$L = \sum_{d \in D} |\mathcal{V}(d)| = \sum_{S_{A_i} \in \mathcal{A}} |A_i|\,|S_{A_i}|$$

vector–user incidences. For Atom S_{A_i}, let $r_i = |\mathcal{I}(S_{A_i})|$ be its number of owner partitions, and let

$$R = \sum_{S_{A_i} \in \mathcal{A}} r_i$$

be the number of Atom placements. Thus R measures logical replication, while $M(\mathcal{P}) = \sum_i r_i |S_{A_i}|$ measures the corresponding vector-copy budget.

Let $H = \sum_{u \in U} |\mathcal{P}(u)|$ be the number of route–partition memberships. For a retained Shared-Atom Graph edge $e = (P_x, P_y)$, define

$$\kappa_e = |\mathcal{S}(P_x) \cup \mathcal{S}(P_y)| + |U_e|, \qquad U_e = \{u \mid P_x \in \mathcal{P}(u) \vee P_y \in \mathcal{P}(u)\}.$$

The first term is the Atom state needed to form X, Ω, Y; the second is the number of routes whose local cost or coverage may change. Cardinalities, owner lists, route memberships, and per-route visible counts are maintained in the catalog, inverted table, and routing directory. Consequently, a recall-model evaluation for one routed output partition is $O(1)$, and evaluating all five alternatives for e takes $O(\kappa_e)$ time. The constant five does not appear in the asymptotic bounds.

Each live partition also carries a generation number. The candidate heap and edge cache use endpoint identifiers and generations as keys, so checking a popped candidate's validity is $O(1)$; an accepted transformation invalidates only entries incident to changed endpoints. These version records are included in the $O(|E|)$ heap-and-cache state below and do not alter the asymptotic bounds.

E.2 Atomization and Seed Construction

If every vector's resolved visible-user set is supplied in canonical form, hashing that set and inserting the vector into its Access Atom takes expected $O(L+n)$ time and $O(L+n+m)$ space. A deterministic implementation that sorts the n visibility signatures instead has $O(L + n \log n)$ time. These bounds include reading the resolved authorization result; SQUID does not inspect the policy program that produced it.

The per-user seed creates one logical placement of S_{A_i} for every user in A_i. Its placement count is

$$R_0 = \sum_{S_{A_i} \in \mathcal{A}} |A_i|,$$

so constructing the seed's route and inverted-table entries costs $O(R_0 + |U|)$. Physically materializing the seed writes $L = M(\mathcal{P}^{(0)})$ vector copies and therefore costs $O(L)$, excluding index construction. If the seed indexes are built immediately, the additional cost is $\sum_{P \in \mathcal{P}^{(0)}} T_{\text{build}}(|P|)$.

E.3 Shared-Atom Graph Construction

For each Atom with $r_i > 1$, core-star construction scans its owner list once to select the core and emits $r_i - 1$ raw star incidences. Hence the number of raw incidences is

$$E_{\text{raw}} = \sum_{i=1}^{m} \max\{r_i - 1, 0\} = O(R),$$

and building the owner lists, core choices, and aggregated shared-Atom sets costs $O(R+E_{\text{raw}})$ time. The graph is subsequently pruned to the top-d incident star edges per partition. Sorting the raw incident lists costs

$$O\left(\sum_{P \in \mathcal{P}} \deg_{\text{raw}}(P) \log \deg_{\text{raw}}(P)\right) \subseteq O(E_{\text{raw}} \log p),$$

and the retained graph has

$$|E| \le \min\{E_{\text{raw}}, dp\}.$$

Thus the candidate graph uses $O(R + |E|)$ space rather than the $O(\sum_i r_i^2)$ space of materializing every Atom-owner clique.

For every retained edge, SQUID enumerates the five local transformations, retains the best memory-saving alternative under the cost-priority rule, and pushes it into a heap. Initial candidate evaluation and heap construction therefore cost

$$T_{\text{init-heap}} = O\left(\sum_{e \in E} \kappa_e + |E| \log |E|\right).$$

This bound is output-sensitive: an Atom shared by many partitions contributes linearly through its core star, while the top-d rule bounds the number of costly local evaluations that reach the heap.

E.4 Greedy Partition Planning

Let q be the number of accepted consolidations before the copy budget is met. Every accepted offline transformation removes at least one Atom placement and does not introduce a new placement, so $q \le R_0 - m$. At iteration t, let F_t be the number of retained edges revalidated after replacing the two endpoint partitions, and let $\bar{\kappa}_t$ be the average local-evaluation size among those edges. Popping from the heap, applying the route-preserving replacement, and

updating the affected catalog and route entries cost $O(\kappa_t + \log|E|)$. Refreshing the changed neighborhood costs $O(F_t(\bar{\kappa}_t + \log|E|))$. The logical planning time is therefore

$$T_{\text{plan}} = O\Bigg(L + R + E_{\text{raw}} \log p + \sum_{e \in E} \kappa_e + |E| \log |E|\Bigg) + O\Bigg(\sum_{t=1}^{q} \left[\kappa_t + \log|E| + F_t(\bar{\kappa}_t + \log|E|)\right]\Bigg).$$

Only edges incident to changed output partitions are refreshed; unrelated partitions, routes, and heap entries are retained. A defensive heap rebuild, used only after stale entries exhaust the heap, reconstructs the current core-star graph and has the same asymptotic cost as graph construction plus $T_{\text{init-heap}}$. If b_{rb} such rebuilds occur, the displayed bound gains $b_{\text{rb}}(O(R + E_{\text{raw}} \log p) + T_{\text{init-heap}})$. In the normal feasible-budget path, local refreshes maintain the candidate heap and this fallback is not on the critical path.

In the pessimistic case, a local transformation can affect all retained edges and all affected routes. Taking $F_t = O(|E|)$ and $\kappa_t, \bar{\kappa}_t = O(m + |U|)$ gives a conservative worst-case bound of

$$O(q|E|(m + |U| + \log|E|))$$

beyond initialization. This is intentionally loose: it describes a globally coupled instance, whereas SQUID's sparse graph and incremental refresh are designed to make $F_t \ll |E|$ in practice.

After the logical plan is fixed, physically building its ANN indexes costs

$$T_{\text{materialize}} = O(M(\mathcal{P})) + \sum_{P \in \mathcal{P}} T_{\text{build}}(|P|).$$

The first term writes vector copies, and the second is index-specific. It is reported separately from planning because it is not an operation performed by the Shared-Atom Graph or the recall-aware cost model.

E.5 Query-Time Cost

For a query from user u, let $h_u = |\mathcal{P}(u)|$. The Search Engine first sorts its route by stored selectivity, costing $O(h_u \log h_u)$. It then probes each routed partition once at ef_0, and may issue at most one expanded search at ef_P. With constant-time visibility checks and a size-k result heap, the query cost is bounded by

$$T_{\text{query}}(u) = O\Bigg(h_u \log h_u + \sum_{P \in \mathcal{P}(u)} \left[T_{\text{ann}}(|P|, ef_0) + ef_0 \log k + \mathbf{1}_P \left(T_{\text{ann}}(|P|, ef_P) + ef_P \log k\right)\right]\Bigg),$$

where $\mathbf{1}_P \in \{0, 1\}$ indicates whether the route is expanded. The $ef \log k$ terms account for authorization filtering and merging candidates into the global top-k. The expression deliberately leaves T_{ann} abstract because exact ANN search complexity depends on the selected index and its parameters; SQUID adds only route ordering, filtering, and result merging.

E.6 Incremental-Maintenance Cost

Consider one update batch. Let L_Δ be the number of changed vector–user visibility incidences, R_Δ the Atom placements touched by the affected Atoms, and U_Δ the number of affected routes. Updating resolved visibility, the Atom Catalog, the Atom Inverted Table, and the User Routing Directory costs

$$O(L_\Delta + R_\Delta + U_\Delta),$$

excluding index insertion or deletion work. The one-hop repair region contains $p_\Delta^+ = |\mathcal{P}_\Delta^+|$ partitions and E_Δ retained Shared-Atom edges. If q_Δ local consolidations are accepted and $F_{\Delta,t}$ edges are refreshed after step t, its logical repair cost is

$$T_{\text{repair}} = O\Bigg(\sum_{e \in E_\Delta} \kappa_e + |E_\Delta| \log |E_\Delta| + \sum_{t=1}^{q_\Delta} \left[\kappa_t + \log|E_\Delta| + F_{\Delta,t}(\bar{\kappa}_t + \log|E_\Delta|)\right]\Bigg).$$

This has the same form as offline planning, but depends only on the changed one-hop neighborhood rather than the global graph.

Compaction checks tombstone ratios in $O(p_\Delta^+)$ time. If C_Δ is the set of compacted partitions, rebuilding their live-vector indexes adds

$$\sum_{P \in C_\Delta} \left(O(|P|) + T_{\text{build}}(|P|)\right).$$

For selectivity repair, let $a_P = |\mathcal{S}(P)|$, u_P be the number of users routed to a mixed partition P, and $\sigma_P = \sum_{S_{A_i} \in \mathcal{S}(P)} |A_i|$ be its Atom–user incidence count. Computing filtering impacts uses $O(\sigma_P)$ time. SQUID ranks users and considers $g = \min\{u_P, 64\}$ high-impact seed users in the implementation; testing seed pairs, assigning remaining users, and evaluating the resulting split take

$$O\left(\sigma_P + u_P \log u_P + g^2 a_P + u_P a_P\right).$$

The fixed cap makes the seed-pair term linear in a_P in the implementation. An accepted split additionally writes replicated copies and builds the two output indexes, costing $O(|P_1| + |P_2| + T_{\text{build}}(|P_1|) + T_{\text{build}}(|P_2|))$.

Combining these terms, the non-index portion of a batch is

$$O(L_\Delta + R_\Delta + U_\Delta + T_{\text{repair}} + p_\Delta^+ + \sum_{P \in \mathcal{S}_\Delta} T_{\text{split}}(P)),$$

where $\mathcal{S}_\Delta$ is the set of partitions considered for splitting. Physical index work is added only for new placements, compacted partitions, and accepted split outputs. Thus, unchanged partitions avoid both global candidate regeneration and index rebuilding.

E.7 Space Complexity

The persistent logical state consists of the resolved relation and Atom memberships, the Atom Inverted Table, user routes, and the retained graph. Its space is $O(L + n + m + R + H + |E|)$, in addition to the $O(M(\mathcal{P}))$ physical vector-copy and ANN-index footprint. The planner's candidate heap and edge cache require $O(|E|)$ live entries, with additional transient stale entries reclaimed by heap rebuilding. For an update batch, the auxiliary state is $O(R_\Delta + |E_\Delta| + U_\Delta)$ plus the materialized replacement partitions, rather than a duplicate of the full partition organization.